

数据库系统概论复习+真题+答案

本人是尼吉软件学院的学长，之前做真题的时候，也苦于没有答案校对。

后面和几个满绩的同学讨论和整理了大概七八套真题，结合 ppt、其他高手的复习资料，做了以下资料整理，希望对学弟学妹们有用。

特别鸣谢软件学院的卓新杰、张桁亿同学。

说明：本文内容参考了《数据库系统概论》、老师教材 ppt 和文档、真题部分的答案（笔者参考了几位满绩点同学的答案）仅供参考，如有错误请及时反馈。

目录

一、 数据库基本记背概念

二、 重要考点和题型归纳

1. 关系代数
2. SQL 语句
3. ER 图与设计
4. 范式与分解
5. 锁与调度

三、 历年真题

正文内容：

一、 数据库的基本记背概念

数据库管理系统

- Database Management System (DBMS)
- 数据 + 管理系统

数据库所解决的问题

- 冗余与不一致性
- 获取数据困难
- 数据孤岛
- 完整性问题

数据库的三层抽象

- **物理层：最低层次的抽象**，描述数据实际上是怎样存储的。物理层详细描述复杂的底层数据结构。
- **逻辑层：介于中间，比物理层层次稍高的抽象**，描述数据库中存储什么数据及这些数据间存在什么关系。这样逻辑层就通过少量相对简单的结构描述了整个数据库。虽然逻辑层的简单结构的实现可能涉及复杂的物理层结构，但逻辑层的用户不必知道这样的复杂性。这称作**物理数据独立性**(physical data independence)。**数据库管理员 (DBA) 使用抽象的逻辑层**，他必须确定数据库中应该保存哪些信息。
- **视图层：最高层次的抽象**，只描述整个数据库的某个部分。尽管在逻辑层使用了比较简单的结构，但由于一个大型数据库中所存信息的多样性，仍存在一定程度的复

杂性。数据库系统的很多用户并不需要关心所有的信息,而只需要访问数据库的一部分。视图层抽象的定义正是为了使这样的用户与系统的交互更简单。系统可以为同一数据库提供多个视图。

实例与模式

- **实例**: 指特定时刻存储在数据库中的信息的集合
- **模式**: 指数据库的总体设计
 - **物理模式**: 在物理层描述数据库的设计
 - **逻辑模式**: 在逻辑层描述数据库的设计。程序员使用逻辑模式来构造数据库应用程序
 - **子模式**: 描述了数据库的不同视图

数据模型

- **关系模型**: 用表的集合来表示数据和数据间的联系
- **实体-联系模型**: 基于对现实世界的认识——现实世界由一组称作实体的基本对象以及这些对象间的联系构成。广泛用于数据库设计
- **基于对象的数据模型**: 基于面向对象设计思想
- **半结构化数据模型**:

数据库架构

- 集中式
- 客户/服务器式
- 并行式
- 分布式

关系型数据库概述

关系

- **属性**: 表中每一列数据名。A1, A2, ..., An
- **元组**: 表中每一行数据
- **关系**: 关系是无序的
 - **关系实例**: 表
 - **关系模式**: $R = (A_1, A_2, \dots, A_n)$ 。例如: instructor = (ID, name, dept_name, salary)

码/键

- **超码**: 一个或一组属性,能够唯一区分一个关系的任何一个元组。例如 {ID, name}, {ID}
- **候选码**: 最小的(包含属性个数最少)超码。例如 {ID}
- **主码**: 候选码中挑出一个作为主码,任何关系只能有一个主码
- **外码**: 一个表中某一列的所有值一定出现在另一张表的某一列,且在另一张表中为主码

视图

- **定义**:
 - 视图是一种虚拟表,本身是不具有数据的,占用很少的内存空间,它是SQL中的一个重要概念。
 - 视图建立在已有表的基础上,视图赖以建立的这些表称为基表。

- 视图的创建和删除只影响视图本身，不影响对应的基表。但是当对视图中的数据进行增加、删除和修改操作时，数据表中的数据会相应地发生变化，反之亦然。
- 向视图提供数据内容的语句为 `SELECT` 语句，可以将视图理解为**存储起来的 `SELECT` 语句**
- 视图不会保存数据，数据真正保存在数据表中。当对视图中的数据进行增加、删除和修改操作时，数据表中的数据会相应地发生变化；反之亦然。
- **不建议对视图进行修改，一般只用于查询**
- 优点：
 - **操作简单**：将经常使用的查询操作定义为视图，可以使开发人员不需要关心视图对应的数据表的结构、表与表之间的关联关系，也不需要关心数据表之间的业务逻辑和查询条件，而只需要简单地操作视图即可，极大简化了开发人员对数据库的操作。
 - **减少数据冗余**：视图跟实际数据表不一样，它存储的是查询语句。所以，在使用的时候，我们要通过定义视图的查询语句来获取结果集。而视图本身不存储数据，不占用数据存储的资源，减少了数据冗余。
 - **数据安全**：MySQL 将用户对数据的访问限制在某些数据的结果集上，而这些数据的结果集可以使用视图来实现。用户不必直接查询或操作数据表。这也可以理解为视图具有隔离性。视图相当于在用户和实际的数据表之间加了一层虚拟表。同时，MySQL 可以根据权限将用户对数据的访问限制在某些视图上，**用户不需要查询数据表，可以直接通过视图获取数据表中的信息**。这在一定程度上保障了数据表中数据的安全性。
 - **适应灵活多变的需求** 当业务系统的需求发生变化后，如果需要改动数据表的结构，则工作量相对较大，可以使用视图来减少改动的工作量。这种方式在实际工作中使用得比较多。
 - **能够分解复杂的查询逻辑** 数据库中如果存在复杂的查询逻辑，则可以将问题进行分解，创建多个视图获取数据，再将创建的多个视图结合起来，完成复杂的查询逻辑。
- 不足：
 - **难以维护**：如果实际数据表的结构变更了，我们就需要及时对相关的视图进行相应的维护。特别是嵌套的视图（就是在视图的基础上创建视图），维护会变得比较复杂，可读性不好，容易变成系统的潜在隐患。因为创建视图的 SQL 查询可能会对字段重命名，也可能包含复杂的逻辑，这些都会增加维护的成本。

实际项目中，如果视图过多，会导致数据库维护成本的问题。

函数

- 定义：
 - MySQL 支持自定义函数，定义好之后，调用方式与调用 MySQL 预定义的系统函数一样。
 - **有返回值**
 - 用户自己定义的存储函数与 MySQL 内部函数是一个性质的。区别在于，存储函数是用户自己定义的，而内部函数是 MySQL 的开发者定义的。

- 参数类型：总是默认为 IN 参数。
- RETURNS 子句只能对 FUNCTION 做指定，对函数而言这是**强制的**。它用来指定函数的返回类型，而且函数体必须包含一个 RETURN value 语句。
- **存储函数可以放在查询语句中使用，存储过程不行**

存储过程

- 定义：
 - 存储过程是一组经过**预先编译的 SQL 语句**的封装。预先存储在 MySQL 服务器上，需要执行的时候，客户端只需要向服务器端发出调用存储过程的命令，服务器端就可以把预先存储好的这一系列 SQL 语句全部执行。
 - 直接操作数据库底层数据结构，一般用于进行更复杂的数据处理
 - **没有返回值**
 - 参数类型：
 - IN：当前参数为输入参数，也就是表示入参；默认为 IN。
 - OUT：当前参数为输出参数，也就是表示出参。
 - INOUT：当前参数既可以为输入参数，也可以为输出参数。
 - 存储过程的功能更加强大，包括能够执行对表的操作（比如创建表，删除表等）和事务操作，这些功能是存储函数不具备的。

数据库设计的目标

令 R 表关系模式，F 为 R 上的函数依赖。

首先，判断 R 是否是一个好的范式。

如果不是，需要对 R 进行分解，将 R 分解为 {R1, R2, ..., Rn}。

- (1) 分解后的 R1, R2, ..., Rn 必须都是好的范式；
- (2) 分解是无损的；
- (3) 最好还能保持函数依赖

ER 模型中的三个基本概念

- **实体集**
 - **实体**：是一个存在的物体，且区别于其他物体。实体可以表示为一组属性的集合
 - **实体集**：是具有相同类型，共享相同属性的实体的集合。
 - 一个或一些属性形成**主键**可唯一区分实体集中的每一个实体
 - **强实体集**：有主码的实体集，通过主码区分每一个实体
 - **弱实体集**：没有足够的属性以形成主码的实体集
 - 每个弱实体集**必须**与一个**标识实体集**（强实体集）建立联系。
 - 与弱实体集建立联系的联系集叫做**标识性联系**。
 - 虽然弱实体集没有主码，但是我们仍需要区分依赖于特定强实体集的弱实体集中的实体的方法。弱实体集的**分辨符**是使得我们进行这种区分的属性集合。
 - 弱实体集的主码由标识实体集的主码加上该弱实体集的分辨符构成。

数据库中的事务

事务是一组不可分割的操作单元，它们作为一个整体被执行，以保证数据库的完整性和一致性。事务具有以下四个重要的属性，通常被称为 **ACID 属性**：

1. **原子性 (Atomicity)**：事务中的所有操作要么全部完成，要么全部不完成，不会结束在中间某个点。这确保了数据库状态的一致性。
2. **一致性 (Consistency)**：事务必须保证数据库从一个一致的状态转移到另一个一致的状态。这意味着在事务开始之前和事务提交之后，数据库的完整性约束必须得到满足。
3. **隔离性 (Isolation)**：并发执行的事务之间不会互相影响。每个事务都像是在独立操作数据库一样，尽管多个事务可能同时进行。
4. **持久性 (Durability)**：一旦事务提交，它对数据库的改变就是永久性的，即使系统发生故障也不会丢失这些改变。

事务的状态

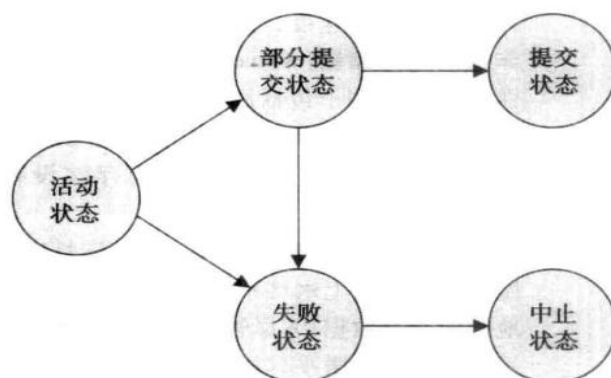


图 14-1 事务状态图

- **活动的**：初始状态，事务执行时处于这个状态。
- **部分提交的**：最后一条语句执行后。
- **失败的**：发现正常的执行不能继续后。
- **中止的**：事务回滚并且数据库已恢复到事务开始执行前的状态后。
- **提交的**：成功完成后。

冲突可串行化调度

- 什么调度是正确的？

串行调度、可串行化调度。

- **可串行化调度**：多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同，称这种调度策略为可串行化调度。

优先图判断冲突可串行化

根据不同事物对同一资源的读写顺序画出优先图，如果没有环路，则冲突可串行化。

并发控制与锁

排他锁：如果当前指令对数据项获得排他锁，则该指令对数据项既可以读也可以写。

共享锁：如果当前指令对数据项获得共享锁，则该指令对数据项只能进行读操作。

许多事务可以同时持有一个数据项上的共享锁，但是只有当其他事务在一个数据项上不持有任何锁（无论共享锁或排他锁）时，一个事务才允许持有该数据项上的排他锁。

两阶段封锁协议：该协议要求每个事务分两个阶段提出加锁和解锁申请。

- **增长阶段：**事务可以获得锁，但不能释放锁。
- **缩减阶段：**事务可以释放锁，但不能获得新锁。

严格两阶段封锁协议：这个协议除了要求封锁是两阶段之外，还要求事务持有的**所有排他锁必须在事务提交后方可释放**。这个要求保证未提交事务所写的任何数据在该事务提交之前均以排他方式加锁，防止其他事务读这些数据。**可以避免级联回滚**。

强两阶段封锁协议：要求事务提交之前不能释放任何锁。

锁转换：

- **升级：**共享锁 → 排他锁，只能发生在增长阶段。
- **降级：**排他锁 → 共享锁，只能发生在缩减阶段。

数据库中的死锁（Deadlock）是指两个或多个事务（Transaction）在执行过程中，因争夺资源而造成的一种相互等待的现象。在这种情况下，每个事务都在等待其他事务释放资源，而由于每个资源都被某个事务持有，导致没有事务能够继续向前推进，形成了一个循环等待的状态。

死锁通常涉及以下四个条件，这四个条件同时满足时就可能发生死锁，它们被称为死锁的四个必要条件：

1. **互斥条件（Mutual Exclusion）：**资源不能被多个事务共享，一次只能由一个事务占有。
2. **占有和等待条件（Hold and Wait）：**一个事务至少占有一个资源，并等待获取其他事务持有的资源。
3. **不可剥夺条件（No Preemption）：**已经分配给一个事务的资源，在事务使用完之前不能被强行夺走。
4. **循环等待条件（Circular Wait）：**存在一个事务序列，其中每个事务都在等待下一个事务所占有的资源。

解决死锁的方法通常包括：

- **预防：**通过破坏死锁的四个必要条件之一或多个来避免死锁的发生。
- **检测：**允许死锁发生，然后通过某种机制检测到死锁，并采取措施解决。
- **避免：**使用算法动态地避免死锁的发生，例如银行家算法。
- **超时：**事务等待资源超过一定时间后超时，释放已占有的资源并回滚。

在数据库管理系统中，通常有专门的死锁检测机制，当检测到死锁时，系统会选择一些事务进行回滚，以释放资源供其他事务使用，从而解决死锁问题。此外，通过合理的事务调度和资源分配策略，也可以降低死锁发生的概率。

二、重要考点和题型归纳

6. 关系代数
7. SQL 语句
8. ER 图与设计
9. 范式与分解
10. 锁与调度

1、关系代数基本知识

1 选择 (Selection)

$$\sigma_{\text{筛选条件}}(R)$$

- 例子：选择 Students 关系中 Major 为 "Computer Science" 的元组。

$$\sigma_{\text{Major}='ComputerScience'}(Students)$$

2 投影 (Projection)

$$\pi_{A_1, A_2, \dots, A_n}(R)$$

- 例子：从 Students 关系中投影 Name 和 Major 属性。 $\pi_{\text{Name, Major}}(Students)$

3 并 (Union)

$$R_1 \cup R_2$$

- 例子：Students 和 Teachers 两个关系的所有元组的并集。 $Students \cup Teachers$

4 差 (Difference)

$$R_1 - R_2$$

- 例子：Students 减去 female_Students。 $Students - female_Students$

5 笛卡尔积 (Cartesian Product)

$$R_1 \times R_2$$

- 例子：Students 和 Courses 的笛卡尔积，构成一个新的表。 $Students \times Courses$

6 连接 (Join)

$$R_1 \bowtie_{\text{条件}} R_2$$

- 例子：Students 和 Enrollments 基于 ID 的等值连接。 $Students \bowtie_{ID=StudentID} Enrollments$

7 除法 (Division)

$$R_1 \div R_2$$

- 例子：找出所有至少被三个学生选修的课程。 $Courses \div Enrollments$
- 这里 Courses 除以 Enrollments 意味着对于 Enrollments 中的每组学生，找出 Courses 中相应的课程，然后选择那些至少出现三次的课程。

赋值 (Assignment)

赋值操作的一般形式为：

temp_name $\leftarrow$ 关系代数表达式

其中 temp_name 是一个临时关系变量的名称，而关系代数表达式则是任何有效的关系代

数操作表达式。

例如，如果我们想要连接 **Students** 和 **Courses** 两个关系，并且将这个连接操作的结果赋值给一个临时变量 **temp**，我们可以这样写：

temp $\leftarrow$ **Students** $\bowtie$ **Courses**

然后，我们可以在其他操作中使用 **temp** 这个变量，比如：

result $\leftarrow \pi_{\text{Students} \cup \text{Courses}} (\sigma_{\text{temp.Sal} > 2000}(\text{temp}))$

8 重命名 (Renaming)

$\rho_{\text{new}}(\text{old})(R)$

- 例子: 将 **Students** 中的 **Name** 列重命名为 **FullName**。 $\rho_{\text{FullName}}(\text{Name})(\text{Students})$

10 分组 (Grouping)

GROUP BY A(R)

- 例子: 按 **Major** 分组并计算每个专业的学生人数。 **GROUP BY Major (Students)**

题型模拟：

1、已知表如下，回答下面问题：

ID	NAME	DEPR_NAME	SALARY
76766	Crick	Bi logy	72000
45565	Katz	Comp. Sci.	75000
12121	Wu	Finance	80000
98345	Kim	Physics	68200

(1) 选出物理系的所有老师

$\sigma_{\text{dept_name} = \text{"Physics"}}(\text{instructor})$

(2) 选出物理系或者年薪资大于 70000 美元的老师；

$\sigma_{\text{dept_name} = \text{"Physics"} \vee \text{salary} > 70000}(\text{instructor})$

(3) 选出除了物理和计算机学院之外的老师

$\sigma_{\neg \text{dept_name} = \text{"Physics"} \wedge \neg \text{dept_name} = \text{"Comp. Sci."}}(\text{instructor})$

(4) 输出老师的 ID, 姓名和工资

$\Pi_{\text{ID}, \text{name}, \text{salary}}(\text{instructor})$

(4) 输出计算机系老师的 ID, 姓名和工资

$\Pi_{\text{ID}, \text{name}, \text{salary}}(\sigma_{\text{dept_name} = \text{"Comp. Sci."}}(\text{instructor}))$

(6) 查询出在计算机学院或者物理学院工作的老师的信息

$\sigma_{dept_name='Physics'}(instructor) \cup \sigma_{dept_name='Physics'}(instructor)$

(7) 找到在 2018 年秋季学期、2019 年春季学期或这两个学期都开设的所有课程。

$\Pi_{course_id} (\sigma_{semester='Fall' \wedge year=2018}(section)) \cup$
 $\Pi_{course_id} (\sigma_{semester='Spring' \wedge year=2019}(section))$

(8) 找到所有在物理学院工作但是工资小于 70000 的所有老师。

$\sigma_{dept_name='Physics'}(instructor) -$
 $\sigma_{salary > 70000}(instructor)$

(9) 找到计算机系且工资大于 70000 的所有老师的工号。

$\Pi_{ID} (\sigma_{dept_name='Comp. Sci.'}(instructor)) \cap \Pi_{ID} (\sigma_{instructor.salary > 70000}(instructor))$

(10) 不使用聚合函数，查出这些老师里的最高工资。

$\Pi_{salary}(instructor) - \Pi_{instructor.salary} (\sigma_{instructor.salary < d.salary}(instructor \times \rho_d(instructor)))$

真题演练：

已知银行企业的数据库由以下表组成：

分行表 branch(branch-name, branch-city, assets)

客户表 customer(customer-name, customer-street, customer-city)

贷款明细表 loan(loan-number, branch-name, amount)

客户贷款表 borrower(customer-name, loan-number)

日存款明细表 account(account-number, branch-name, balance)

客户存款表 depositor(customer-name, account-number)

注：带下划线的属性为主码，假设客户的名字不相同

(1) 用 SQL 语句创建表。

(2) 使用关系代数和 SQL 语句找出在 Brighton 银行中有存款的所有客户的姓名存款号和存款额。

(3) 使用关系代数和 SQL 语句找出账户平均余额小于 5000 元的支行，显示支行名称及账户平均余额。

(4) 使用关系代数和 SQL 语句找出所有在银行中有贷款但无账户的客户

(5) 使用关系代数和 SQL 语句对所有存款余额大于平均存款额的账户付 3% 的利息。

(1) 用 SQL 语句创建表。

```
create table branch(
branch-name char(15) primary key
branch-city char(30)
assets numeric(16,2)
primary key(branch-name));
```

(2) 使用关系代数和 SQL 语句找出在 Brighton 银行中有存款的所有客户的姓名、存款号、和存款额。

$\Pi_{customer-name, depositor.account-number, balance} (\sigma_{depositor.account-number=account.account-number \wedge branch-name='Brighton'}(depositor \times account))$

Sql 语句：

```
select customer-name, depositor. account-number, balance
from depositor, account
where depositor.account-number=account.account-numberand
```

and branch-name='Brighton' :

3) 使用关系代数和 SQL 语句找出账户平均余额小于 5000 元的支行, 显示支行名称及账户平均余额

$$\sigma_{ab < 5000}(\rho_{branch=balance(branch-name, ab)}(branch-name \mathcal{G}_{avg(balance)}(account)))$$

```
select branch-name, avg(balance)
from account
group by branch-name
having avg(balance) < 5000
```

(4) 使用关系代数和 SQL 语句找出所有在银行中有贷款但无账户的客户。

$$\prod_{customer-name}(borrower) - \prod_{customer-name}(depositor)$$

```
select distinct customer-name
from borrower
where customer-name not in
(select customer-name from depositor)
```

(5) 使用关系代数和 SQL 语句对所有存款余额大于平均存款额的账户增加 3% 的利息。

$$account \leftarrow \prod_{account-number, branch-name, balance * 1.03}(\sigma_{balance > avg(balance)}(account)) \\ \cup \prod_{account-number, branch-name, balance}(\sigma_{balance \leq avg(balance)}(account))$$

```
update account
set balance=balance*1.03
where balance > (select avg(balance) from account)
```

2、SQL 语句

1. 选择 (SELECT)

通式: SELECT [DISTINCT] 列名列表 FROM 表源 [WHERE 条件] SELECT [DISTINCT] 列名列表 FROM 表源 [WHERE 条件]

示例: 选择 Students 表中的 Name 和 Age 列, 其中 Age 大于 20。

```
SELECT DISTINCT Name, Age FROM Students WHERE Age > 20;
```

2. 插入 (INSERT)

通式: INSERT [INTO] 表名 [(列名列表)] VALUES (值列表) INSERT [INTO] 表名 [(列名列表)] VALUES (值列表)

示例: 向 Students 表中插入新记录。

INSERT INTO Students (Name, Age) VALUES ('John Doe', 22);

3. 更新 (UPDATE)

通式: UPDATE 表名 SET 要更新的列名=值 [WHERE 条件] UPDATE 表名 SET 要更新的列名=值 [WHERE 条件]

示例: 更新 Students 表中名为 'John Doe' 的记录, 将其年龄增加 1 岁。

UPDATE Students SET Age = Age + 1 WHERE Name = 'John Doe';

4. 删除 (DELETE)

通式: DELETE [FROM] 表名 [WHERE 条件] DELETE [FROM] 表名 [WHERE 条件]

示例: 删除 Students 表中名为 'John Doe' 的记录。

DELETE FROM Students WHERE Name = 'John Doe';

5. 聚合函数 (Aggregate Functions)

如 COUNT, AVG, SUM, MIN, MAX 等。

通式: SELECT 聚合函数(列名) FROM 表名 [WHERE 条件] SELECT 聚合函数(列名) FROM 表名 [WHERE 条件]

示例: 计算 Students 表中所有学生的最小年龄。

SELECT MIN(Age) FROM Students;

6. 排序 (ORDER BY)

通式 SELECT ... FROM ... [WHERE ...] ORDER BY 列名 [ASC | DESC] SELECT ... FROM ... [WHERE ...] ORDER BY 列名 [ASC | DESC]

示例: 选择 Students 表中的所有记录, 并按 Age 升序排序。

SELECT * FROM Students ORDER BY Age ASC;

7. 分组 (GROUP BY)

通式: SELECT ... FROM ... GROUP BY 列名 [HAVING 条件] SELECT ... FROM ... GROUP BY 列名 [HAVING 条件]

示例: 按 Major 列分组并计算每个专业的学生数量。

SELECT Major, COUNT(*) FROM Students GROUP BY Major;

8. 连接 (JOIN)

包括 INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN。

通式: SELECT ... FROM 表 1 [JOIN 类型] 表 2 ON 连接条件 SELECT ... FROM 表 1 [JOIN 类型] 表 2 ON 连接条件

示例: 内连接 Students 和 Courses 表, 基于 StudentID 和 CourseID。

SELECT Students.Name, Courses.Title

FROM Students

INNER JOIN Enrollments ON Students.ID = Enrollments.StudentID

INNER JOIN Courses ON Enrollments.CourseID = Courses.ID;

9. 子查询 (Subquery)

通式: 子查询可以出现在 SELECT, FROM, WHERE 等子句中。

示例: 在 SELECT 语句中使用子查询来找出分数最高的学生的名字。

SELECT Name FROM Students

WHERE Score = (SELECT MAX(Score) FROM Students);

10. 创建表 (CREATE TABLE)

通式: CREATE TABLE 表名 (列定义, ...) CREATE TABLE 表名 (列定义, ...)

示例： 创建一个名为 Students 的新表。

```
CREATE TABLE Students (  
    ID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT  
);
```

11. 约束 (Constraints)

如 PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK 等。

通式： 在 CREATE TABLE 或 ALTER TABLE 语句中定义约束。

示例： 在创建表时定义主键和外键。

```
CREATE TABLE Students (  
    ID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT,  
    CourseID INT,  
    FOREIGN KEY (CourseID) REFERENCES Courses(ID)  
);
```

12. 聚合函数 (详细)

聚合函数通常与 SELECT 语句结合使用，并且可以搭配 GROUP BY 子句来对数据进行分组统计。

(1) . COUNT

通式： SELECT COUNT(* | 列名) FROM 表名 [WHERE 条件] SELECT COUNT(* | 列名) FROM 表名 [WHERE 条件]

- 用途：计算表中行数或非空值的数量。
- 示例： 计算 Students 表中的总学生人数。

```
SELECT COUNT(*) FROM Students;
```

(2) . AVG

通式： SELECT AVG(列名) FROM 表名 [WHERE 条件] SELECT AVG(列名) FROM 表名 [WHERE 条件]

- 用途：计算数值列的平均值。
- 示例： 计算 Students 表中所有学生的平均年龄。

```
SELECT AVG(Age) FROM Students;
```

(3) . SUM

通式： SELECT SUM(列名) FROM 表名 [WHERE 条件] SELECT SUM(列名) FROM 表名 [WHERE 条件]

- 用途：计算数值列的总和。
- 示例： 计算 Orders 表中所有订单的总金额。

```
SELECT SUM(Amount) FROM Orders;
```

(4) . MIN

通式： SELECT MIN(列名) FROM 表名 [WHERE 条件] SELECT MIN(列名) FROM 表名 [WHERE 条件]

- 用途：找出数值列的最小值。
- 示例： 找出 Students 表中年龄最小的学生。

```
SELECT MIN(Age) FROM Students;
```

(5) . MAX

通式： SELECT MAX(列名) FROM 表名 [WHERE 条件] SELECT MAX(列名) FROM 表名 [WHERE 条件]

- 用途：找出数值列的最大值。
- 示例：找出 Students 表中年龄最大的学生。

SELECT MAX(Age) FROM Students;

(6) . GROUP BY 与聚合函数结合使用

当需要对分组数据分别应用聚合函数时，GROUP BY 子句就非常有用。

通式： SELECT 列名 [, 聚合函数(列名)] FROM 表名 GROUP BY 分组依据列名 [HAVING 条件] SELECT 列名 [, 聚合函数(列名)] FROM 表名 GROUP BY 分组依据列名 [HAVING 条件]

- 用途：根据一个或多个列对结果集进行分组，并可选择性地对每组应用聚合函数。
- 示例：按 Major 列分组，并计算每个专业的学生人数。

SELECT Major, COUNT(*) AS Number_of_Students FROM Students GROUP BY Major;

(7) . HAVING

HAVING 子句在 GROUP BY 之后使用，用于过滤分组后的结果。

通式： SELECT ... FROM ... GROUP BY ... HAVING 条件

SELECT ... FROM ... GROUP BY ... HAVING 条件

- 用途：对分组后的结果进行条件过滤。
- 示例：找出学生人数超过 10 人的每个专业。

SELECT Major, COUNT(*) AS Number_of_Students FROM Students GROUP BY Major HAVING COUNT(*) > 10;

注意：所有的聚合函数（除了 count (*)）都忽略 NULL

select sum (salary) from instructor

在计算 sum 的时候，会忽略 null 值，计算非 null 值的和。如果全部是 null，那么最终这个查询返回的是 null。

同理，max(salary)，min(salary)，avg(salary)也是忽略 null 的；当所有值都是 null，则返回 null。

count(salary)，若 salary 全都是 null，则返回 0。否则返回非 null 的数量。

count (*) 计算的是元组的数量，null 值不忽略。

题型模拟：

1、已知表 instruct 如下，回答下面问题：

ID	NAME	DEPR_NAME	SALARY
76766	Crick	Bilogy	72000
45565	Katz	Comp. Sci.	75000
12121	Wu	Finance	80000
98345	Kim	Physics	68200

- (1) 从 instructor 表中输出 dept_name 的信息，删除重复

select distinct dept_name from instructor

- (2) 查询所有老师的工号、姓名和月薪

select ID, name, salary/12 from instructor

select ID, name, salary/12 as monthly_salary from instructor

- (3) 查询表中 Comp. Sci 系的所有工资大于 8000 元的老师名字。

select name from instructor where dept_name = 'Comp. Sci.' and salary > 80000

- (4) 找出艺术系所有曾经教授过课程的教师的姓名和课程编号 (使用笛卡尔积)
- ```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID and instructor.dept_name = 'Art'
```
- (5) 输出每个系的老师的平均年薪
- ```
select dept_name, avg (salary) as avg_salary
from instructor group by dept_name;
```
- (6) 输出学院内老师的平均年薪大于 40000 的学院和他们的平均工资
- ```
select dept_name, avg (salary)
from instructor
group by dept_name
having avg (salary) > 42000;
```
- (7) 找到比生物学院所有老师工资都高的老师的姓名
- ```
select name
from instructor
where salary > all (select salary
from instructor
where dept name = ' Biology' );
```

真题演练:

1. 有仓库-物料存储系统, 其关系模式如下:

W(Wid, WN, Addr); 分别表示仓库(仓库号, 仓库名称, 地址)

M(Mid, MN, Price, p_Time, q_Time): 分别表示〈物料号, 物料名称, 价格, 生产日期, 保质期)

ST(Wid, Mid, SNum); 分别表示(仓库号, 物料号, 存储数量)

请完成如下查询: (10 分)

1) 用关系代数表达式, 查询存储了全部物料的仓库名称。

$$\Pi_{WN} (W \bowtie \Pi_{Wid} (ST \div \Pi_{Mid} (M)))$$

2) 用关系代数表达式, 查询所有存储数量高于 100 的物料号。

$$\Pi_{mid} \left(\sigma_{Num > 100} \left(\rho_A (Mid, Num) (Mid g_{sum(sNum)} (ST)) \right) \right)$$

3) 用 SQL 语句, 查询所有没有生产日期的物料号和物料名称。

```
SELECT Mid, MN
FROM M
WHERE p_Time IS NULL;
```

4) 用 SQL 语句, 建立视图 V, 统计截止到今天, 还有三个月就要到保质期的物料号和物料名称。

```
CREATE VIEW V AS
```

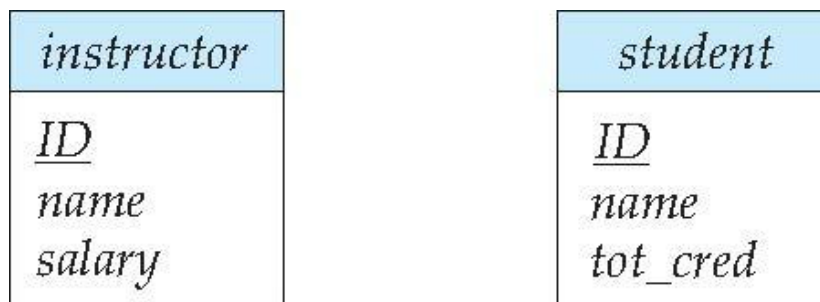
```

SELECT Mid, MN
FROM M
WHERE q_Time <= CURRENT_DATE + INTERVAL '3 MONTH';
5) 用 SQL 语句，修改物料名为“海绵”的库存，增加 100 件。
UPDATE ST
SET SNum = SNum + 100
WHERE Mid = (SELECT Mid FROM M WHERE MN = '海绵');

```

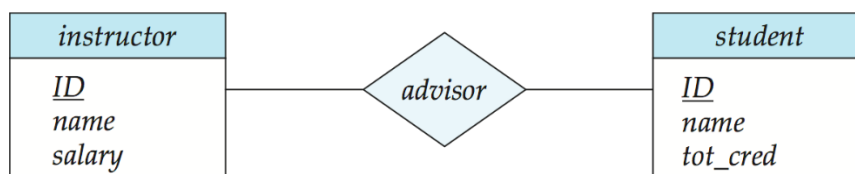
3、ER 图与设计

实体集用两个矩形表示，
 实体集的名字写在上面的矩形，
 属性写在下面的矩形里。
 主码用下划线标出

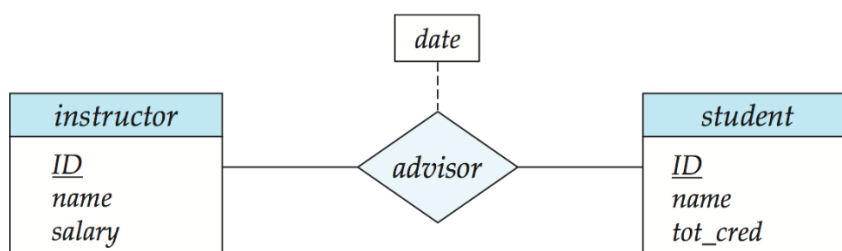


联系集

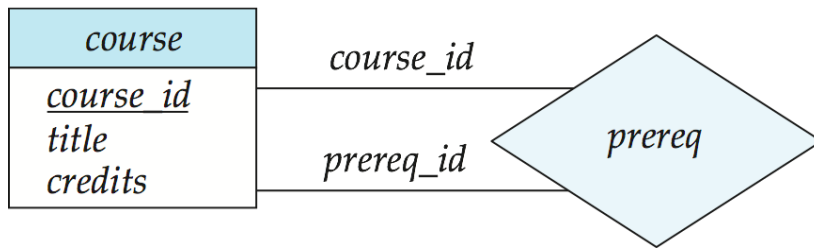
联系集用菱形表示



联系集上的属性



角色

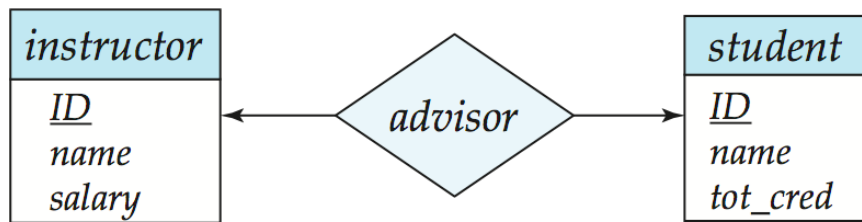


基数约束

参与联系的实体集上需要给出基数约束。

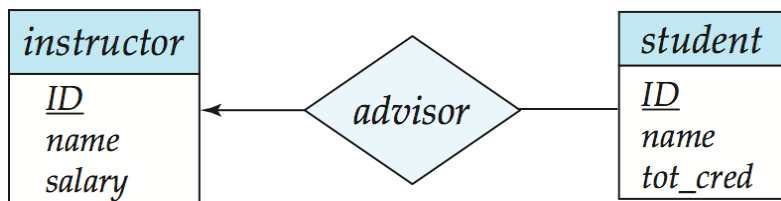
可以用→表示“1”，用无向线段表示“多”

- 一对一



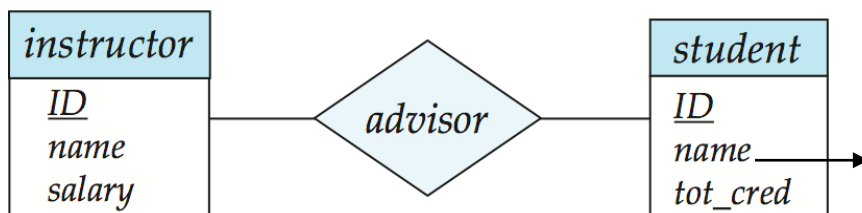
这个图表示 1 个老师只能指导 1 个学生，1 个学生只能被 1 个老师指导。

- 一对多



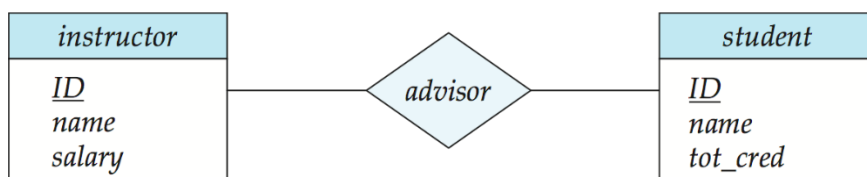
1 个老师可以指导多个学生，1 个学生只能被 1 个老师指导。

- 多对一



1 个老师只能指导 1 个学生，1 个学生可以被多个老师指导。

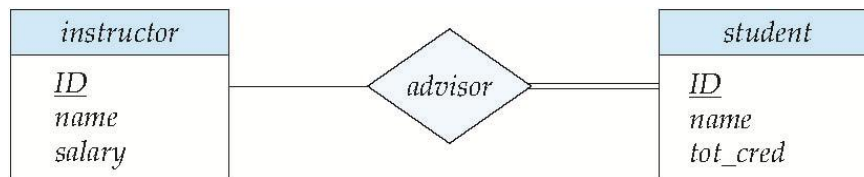
- 多对多



1 个老师可以指导多个学生，1 个学生可以被多个老师指导。

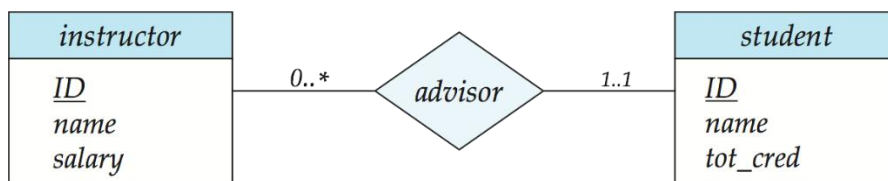
全参与和部分参与

单线表示部分参与，双线表示全参与



老师可以不指导学生，但是每位学生必须至少由 1 名指导老师

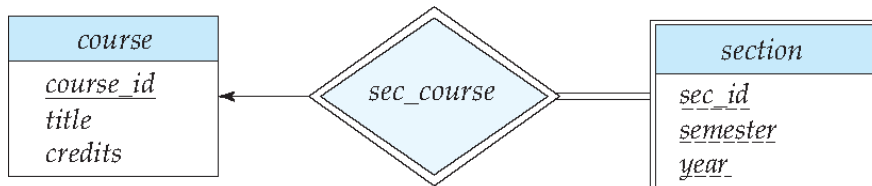
还可以使用 1..h 来表示实体集的基数约束



表示 1 名老师可以指导 0 名或者多名学生，1 名学生只能被 1 名老师指导。

弱实体集

弱实体集用双线矩形表示，说实体集的区别属性集用虚线标记。弱实体集没有足够的属性作为主码，他必须依赖与 1 个强实体集。



3.4 将 E-R 转为关系模式

(1) 强实体集可以直接转化

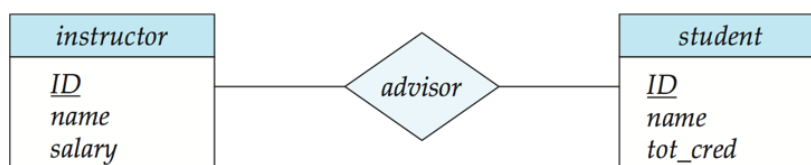
student (ID, name, tot_cred)

(2) 弱实体集除了包含自身的属性，还包括其依赖的强实体集的主码。

section (course_id, sec_id, sem, year)

(3) 联系集包含了自身的属性，还需要包含参与该联系的属性集的主码

advisor = (s_id, i_id)



范式与分解

范式是评价数据库模式规范化程度从低到高主要有：1NF、2NF、3NF、BCNF、4NF、5NF。

数据库范式的作用：

数据库范式主要是为解决关系数据库中数据冗余、更新异常、插入异常、删除异常问题而引入的设计理念。数据库范式可以避免数据冗余，减少数据库的存储空间，并且减轻维护数据完整性的成本。

常考点：

Armstrong 定理

- (1) 自反律: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
- (2) 增强律: $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$
- (3) 传递率: $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$

由上三个定律可以推导出：

- (1) 合并律
 $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$, then $\alpha \rightarrow \beta \gamma$
- (2) 分解律
 $\alpha \rightarrow \beta \gamma$, then $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$
- (3) 伪传递律
 $\alpha \rightarrow \beta$ and $\gamma \beta \rightarrow \delta$, then $\alpha \gamma \rightarrow \delta$

函数依赖

令 R 表示一个关系模式

$$\alpha \subseteq R \text{ 且 } \beta \subseteq R$$

函数依赖 $\alpha \rightarrow \beta$ 在 R 上成立，当且仅当任何一个合法的关系 $r(R)$ 中，如果两个元组 t_1 和 t_2 在 r 中具有相同的 α 的取值，也必须在 β 上有相同的取值，用数学语言表示为：

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- K 是关系模式 R 的超键，当且仅当 K 决定了 R 中的所有属性。
- K 是 R 的候选键，当且仅当：
 - K 决定了 R 中的所有属性，并且对于 K 的任何一个真子集 α ， α 都不能决定 R 中的所有属性。
- 平凡的函数依赖：如果 β 是 α 的子集，那么 α 决定 β 是平凡的函数依赖。（比如

说 $AB \rightarrow B$, 那么它就是平凡的函数依赖)

函数依赖的闭包

所有能够被 F 逻辑蕴含的函数依赖的集合称之为 F 的闭包。(即那些 F 成立, 则其必然也成立的函数依赖的集合)。

闭包用 F^+ 表示。 F^+ 是 F 的超集。

第一范式

如果关系 R 中的所有属性都是原子的, 则该范式满足第一范式。

第三范式

一个关系模式 R 是第三范式, 当所有的在 F 中的 $\alpha \rightarrow \beta$ 至少满足如下三个条件之一:

- (1) $\alpha \rightarrow \beta$ 是平凡的;
- (2) α 是关系模式 R 的一个超码;
- (3) $\beta - \alpha$ 中每个属性 A 均属于某个候选码, 但不要求这些属性同属于一个候选码。

满足 BCNF 一定满足第三范式。第三范式是 BCNF 的最小松弛。

BC 范式:

F^+ 中的任何一个 $\alpha \rightarrow \beta$ 要么是平凡的, 要么 α 是超码。只有满足上述条件, 该关系模式 R 才满足 BCNF 范式。

■判断 F 中的 $\alpha \rightarrow \beta$ 中是否存在无关属性:

若 $A \in \alpha$, A 是无关属性, 当 F 能够蕴含 $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$ 。

若 $A \in \beta$, A 是无关属性, 当 $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ 能够蕴含 F 。

举例: $F = \{A \rightarrow C, AB \rightarrow C\}$

B 在 $AB \rightarrow C$ 中是多余的, 因为 $\{A \rightarrow C, AB \rightarrow C\}$ 在逻辑上蕴含了 $A \rightarrow C$ (即从 $AB \rightarrow C$ 中删除 B 的结果)。

-举例: 给定 $F = \{A \rightarrow C, AB \rightarrow CD\}$,

C 在 $AB \rightarrow CD$ 中是多余的, 因为即使删除 C , 仍然可以推断出 $AB \rightarrow C$ 。

■属性集合的闭包

给定一个属性集合 α , 在函数依赖集合 F 下的 α 的闭包是能由 α 函数依赖所决定的属性的集合。

计算 α 在函数依赖集合 F 下的闭包 α^+

举例:

$R = (A, B, C, G, H, I)$

$F = \{A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$

$(AG)^+$

1. $result = AG$
2. $result = ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
3. $result = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
4. $result = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)

1、设有关系模式 $R = (A, B, C, D, E, G, H)$ ，函数依赖 $F = \{AH \rightarrow C, C \rightarrow A, CH \rightarrow D, C \rightarrow EG, EH \rightarrow C, CG \rightarrow DH, CE \rightarrow AG, ACD \rightarrow H\}$ 。令 $X = BC$ ，求 X 关于 F 的闭包 X^+ ；

【解答】

$result = BC$

$result = BCAEG$ ($C \rightarrow A, C \rightarrow EG$)

$result = BCAEGDH = R$ ($CG \rightarrow DH$) 因此 $X^+ = R$

■求候选码的简单方法：

- (1) 如果有属性不在函数依赖集中出现，那么它必须包含在候选码中；
- (2) 如果有属性只在函数依赖集的右边出现，则该属性一定不包含在候选码中。
- (3) 如果有属性只在函数依赖集的左边出现，则该属性一定包含在候选码中。
- (4) 如果有属性或属性组能唯一标识元组，则它就是候选码；

举例：

1. $R \langle U, F \rangle, U = (A, B, C), F = \{AB \rightarrow C, C \rightarrow B\}$ ，求候选码。

【解答】因为 A 只出现在左边， A 的闭包还是 A ，则对 A 进行组合，可以和 B, C 进行组合。

AB 本身自包 AB ，而 $AB \rightarrow C$ ，所以 $(AB)^+ = ABC = U$ 。

AC 本身自包 AC ，而 $C \rightarrow B$ ，所以 $(AC)^+ = ABC = U$ 。

故候选码是 AB, AC 。

2. 设有关系模式 $R(A, B, C, D, E, F, G)$

其函数依赖集 $F = \{AB \rightarrow C, B \rightarrow DE, C \rightarrow G, G \rightarrow A\}$

求出模式 R 的所有候选码) (7 分)

【解答】

F 中所有在左边的属性有 $ABCG$

所有在右边的属性有 CDEGA

那么可以知道：

左部属性 B

右部属性 DE

双部属性 ACG

外部属性 F

首先计算 $(BF)^+ = \{BFDE\}$

然后计算 ABF^+ 、 CBF^+ 、 GBF^+

第一个可以推 $\{ABCDEFG\}$

第二个可以推 $\{ABCDEFG\}$

第三个可以推 $\{ABCDEFG\}$

因此候选码为：ABF, CBF, GBF

■判断范式：

思路很简单，牢记几大范式的定义，照着定义去判断就行。

举例：

设有属于 1NF 的关系模式 $R=(A, B, C, D, E)$ ，R 上的函数依赖集 $F=\{BC \rightarrow AD, AD \rightarrow EB, E \rightarrow C\}$ 。

(1) R 是否属于 3NF? 为什么?

(2) R 是否属于 BCNF? 为什么?

【解答】关系模式 R 的所有候选码为: BC, AD, BE。

(1) 多函数依赖集 F 中，所有的右部属性都在某个候选码的属性当中，显然是 3NF

根据第三范式定义：所有的在 F^+ 中的 $\alpha \rightarrow \beta$ 需至少满足如下三个条件之一：(注意由某定理可知：判断第三范式只需要在 F 上进行，无需在 F^+ 上。)

(1) $\alpha \rightarrow \beta$ 是平凡的；

(2) α 是关系模式 R 的一个超码；

(3) $\beta - \alpha$ 中每个属性 A 均属于某个候选码，但不要求这些属性同属于一个候选码。

很显然，本题满足第三个条件。

(2) $E \rightarrow C$ 是非平凡依赖，并且 E 不是 R 的一个超码，因此 R 不属于 BCNF

BC 范式：

F^+ 中的任何一个 $\alpha \rightarrow \beta$ 要么是平凡的，要么 α 是超码。只有满足上述条件，该关系模式 R 才满足 BCNF 范式。

很显然，本题一个条件都不满足。

锁与调度

冲突：调度中一对连续的动作，它们满足：如果它们的顺序交换，那么涉及的事务中至少有一个事务的行为会改变。

注意：有冲突的两个操作是不能交换次序的，没有冲突的两个事务是可交换的
几种冲突的情况。一般 R 表示读操作，W 表示写操作。 $w_i(X)$ 表示事务 i 对 X 进行写操作。

(1) 不同事务对同一元素的两个写操作是冲突的

$$w_i(X); w_j(X)$$

(2) 不同事务对同一元素的一读一写操作是冲突的

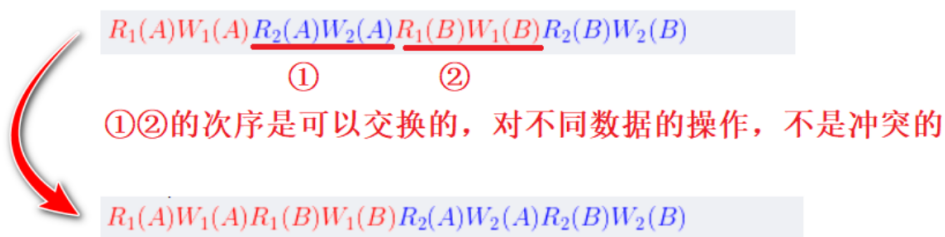
$$w_i(X); r_j(X) \quad r_i(X); w_j(X)$$

(3) 同一事务的任何两个操作都是冲突的

$$r_i(X); w_i(Y)$$

$$w_i(X); r_i(Y)$$

见如下例子：



冲突可串行性判别算法

将调度 S 构造为一个前驱图(有向图、优先图)

结点是每一个事务 T_i 。如果 T_i 的一个操作与 T_j 的一个操作发生冲突，且 T_i 在 T_j 前执行，则绘制一条边，由 T_i 指向 T_j ，表征 T_i 要在 T_j 前执行。

测试检查：若此有向图没有环，则调度 S 是冲突可串行化的。

死锁检测算法（原理和冲突可串行化判别方法一致）

死锁可以用称为等待图的有向图来精确描述。

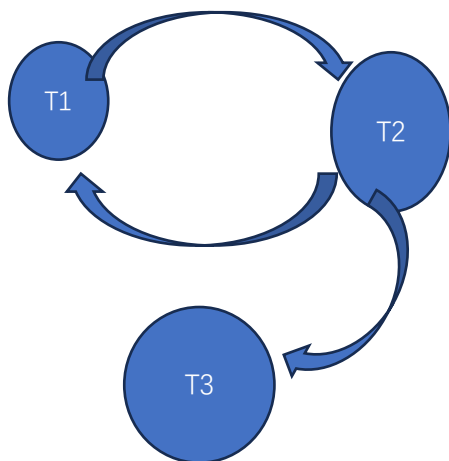
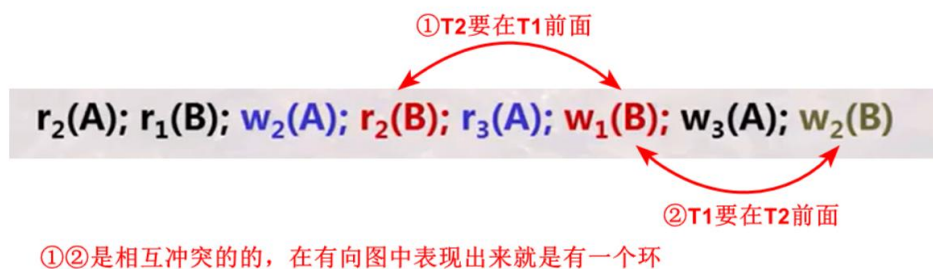
当且仅当等待图包含环时，系统中存在死锁，在该环中的每个事务称为处于死锁。

题型模拟：

判断以下调度 S 是否为冲突可串行化调度：

$r_2(A); r_1(B); w_2(A); r_2(B); r_3(A); w_1(B); w_3(A); w_2(B)$

【解答】



可以看见 T1、T2 成环，因此这个调度不是冲突可串行化调度。

真题演练：

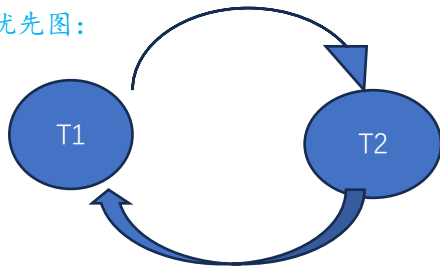
1、判断以下调度 S 是否为串行化调度；如果不是，说明理由。

时序	T1	T2
----	----	----

1	$W(A)$	
2		$W(A)$
3		$R(B)$
4	$W(B)$	

【解答】不是可串行化调度。理由如下：

做出优先图：



可以看出图中有环。

三、历年真题（挑选了七套）

第一套试题（试卷+答案）

考试时间：2021 年 01 月 班级：

一、简答题（共 20 分）

1. DDL 和 DML 的含义分别是什么？

【解答】DDL 指数据定义语言，用于数据模式定义，规定一致性约束；

DML 指数据操作语言，用于数据的增删改查等操作。

2. 在如下 from 子句中：from R, (子查询)，子查询中可以直接使用关系 R 吗？为什么。

【解答】不能，因为这可能会导致存在二义性。

3. 请罗列至少 3 种数据模型（Data Model）

【解答】关系模型；实体_联系模型；半结构化数据模型；层次模型；网状模型；

4. 在 ER 图转化关系模式时候，如何处理联系集？

【解答】当描述的联系映射基数为多对多时，转化为关系模式，主键为所联系的实体集的属性共同构成，同时加上外键约束，增加联系属性。当不为多对多时，则不转化为关系模式。

5. 为什么 3NF 范式分解能够保证函数依赖一定能够得到保持？

【解答】3NF 分解时将每个函数依赖的左右两端属性合并为一个关系，因此在分解成的每个 R_i 中，都一定保持了它所对应的函数依赖。

6. 请简述函数和过程的异同。

【解答】相同点：都是存储在数据库中的代码，封装了一系列操作，都可调用；

不同点：函数必须有显式返回值，过程则不必。函数可以放在查询语句中使用，过程不行。函数不能有 DDL 语言，过程可以。

7. 请用属性集合的闭包来表达属性集合 A 是关系 R 的一个候选码。

【解答】A 在函数依赖 F 上的闭包 $(A)^+ = R$

8. 如果两个关系进行集合并运算，那么这两个关系需要满足哪些条件？

【解答】相同的属性数目，相同的属性顺序，相同的数据类型，相同属性名。

9. 在 SQL 语句中，where 子句可以嵌入子查询。那么该子查询的作用有哪些？（至少两种）

【解答】限定条件：子查询可以用于 WHERE 子句中，为父查询提供特定的过滤条件；存在性检查：子查询可以用于检查某个条件是否至少有一例存在，这通常与 EXISTS 关键字一起使用。

10. 布尔表达式 $\neg(1 > \text{Null}) \wedge (\text{Null} = \text{Null})$ 的结果是什么？

【解答】unknown .

需要注意的是数据库中的 Null 表示的是不确定，因此有 NULL 参与的 bool 运算结果有三种情况。如：3>4 结果为 false；但是对于 3>NULL，因为 NULL 不确定，它可能是大于 3 的数，也可能是小于 3 的数，这就导致 bool 运算的结果不确定，用 unknow 表示。

二、分析题（每题 5 分，共 20 分）

1、请证明分解律：若有函数依赖 $\alpha \rightarrow \beta\gamma$ 成立，则有 $\alpha \rightarrow \beta$, $\alpha \rightarrow \gamma$ 成立。

【解答】

$$\beta \subseteq \beta\gamma \Rightarrow \left. \begin{array}{l} \beta\gamma \rightarrow \beta \\ \text{且} \quad \alpha \rightarrow \beta\gamma \end{array} \right\} \Rightarrow \alpha \rightarrow \beta$$

$$\gamma \subseteq \beta\gamma \Rightarrow \left. \begin{array}{l} \beta\gamma \rightarrow \gamma \\ \text{且} \quad \alpha \rightarrow \beta\gamma \end{array} \right\} \Rightarrow \alpha \rightarrow \gamma$$

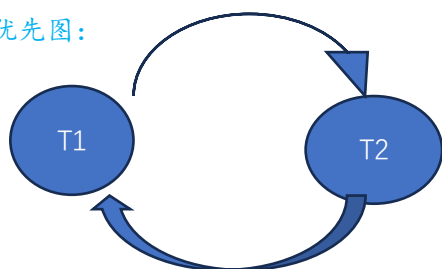
即证。

2、判断以下调度 S 是否为串行化调度；如果不是，说明理由。

时序	T1	T2
1	W(A)	
2		W(A)
3		R(B)
4	W(B)	

【解答】不是可串行化调度。理由如下：

做出优先图：



可以看出图中有环。

3、如果想要查询计算机学院的学生的选课信息，可以有如下两种关系代数表达式；请分析这两种表达式的优劣。

- (1). $\sigma_{\text{dept_name} = \text{'Comp. sci'}}(\text{student} \bowtie \text{takes})$
- (2). $(\sigma_{\text{dept_name} = \text{'Comp. sci'}}(\text{student})) \bowtie \text{takes}$

【解答】前者是连接后再选择，后者是选择后再连接。后者对比前者，缩小了连接过程中产生临时笛卡尔积的大小，加快了连接速度。前者的好处在于思维简单，易于表达。

4、当我们构造 E-R 图的时候，有些实体集自身的属性不足以构成其主码，应该怎么处理？将其转换成关系模式时，其属性集包括哪些内容？

【解答】将该实体集定义为弱实体集，并且以多对一的形式映射，通过弱联系集联系一个强实体集，包括标志性强实体集的主码和自身属性。该弱实体集的主码由标志性强实体集和其自身的分辨符构成。

三、设计题（共 20 分）

吉林大学图书馆欲开发一套管理系统。数据库中需要保存如下信息：

图书：图书编号、书名、价格、作者（图书馆中可能有多本同一种书）

书库：书库号、地点、面积、电话

管理员：工号、姓名、性别、职务

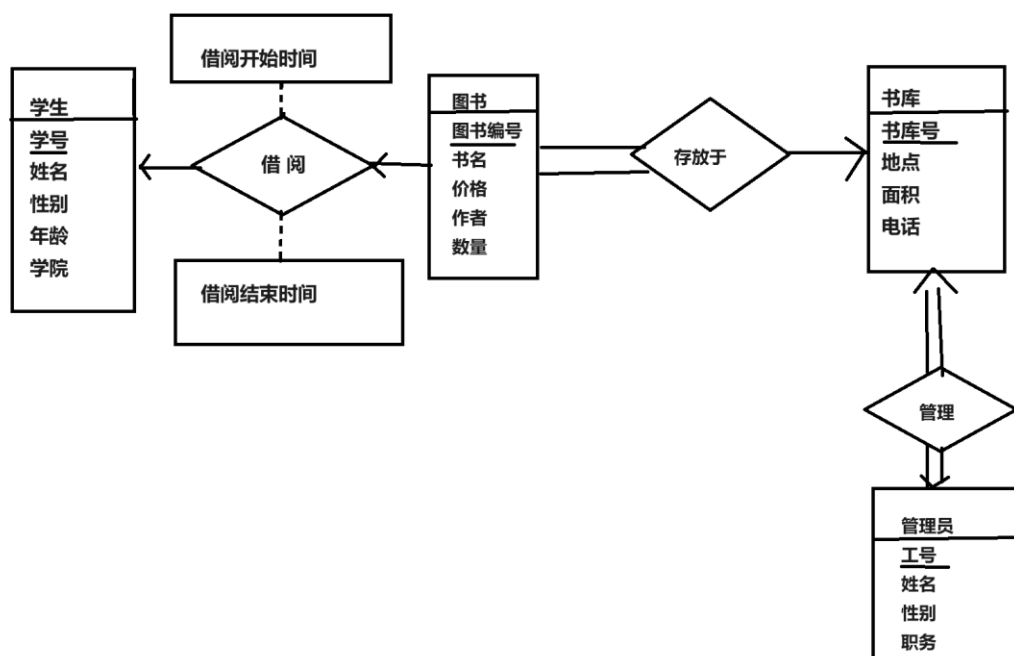
学生：学号、姓名、性别、年龄、学院

其中，一名学生可以借 0 到 5 本图书；借书过程中需要记录借阅起止时间。

完成如下设计：

- (1) 设计该信息管理系统的 E-R 图；
- (2) 将该 E-R 图转换为关系模式；
- (3) 指出转换结果中每个关系模式的主码。

【解答】(1)



(2) 关系模式:

学生 (学号, 姓名, 性别, 年龄, 学院)

图书 (图书编号, 书名, 价格, 作者, 借阅开始时间, 借阅结束时间, 学号, 书库号)

书库 (书库号, 地点, 面积, 电话, 工号)

管理员 (工号, 姓名, 性别, 职务)

(3) 学生主码: 学号; 图书主码: 图书编号; 书库主码: 书库号; 管理员主码: 工号。

四、计算题 (每题 5 分, 共 10 分)

设有关系模式已知关系模式 $R < U, F >$, 若 $U = \{A, B, C, D, E, F\}; F = \{A \rightarrow BCD, BC \rightarrow DE, B \rightarrow D, D \rightarrow A\}$

1. 证明 DF 是一个候选码 (写出关键步骤)

2. 写出所有候选码;

【解答】

1. 要证明 DF 是候选码, 实际上就是要证明 $(DF)^+$ 是不是等于 U

(1) result = DF

(2) result = ADF (因为 $D \rightarrow A$)

(3) result = ABCDF (因为 $A \rightarrow BCD$)

(4) result = ABCDEF (因为 $BC \rightarrow DE$)
故 DF 是候选码。

2.

出现在左边的属性: ABCD

出现在右边的属性: ABCDE

左部属性: 无

右部属性: E

双部属性: ABCD

外部属性: F

$$(AF)^+ = U, (BF)^+ = U, (CF)^+ = CF$$

再结合 (1) 得候选码为 AF, BF, DF

五、应用题 (每题 6 分, 共 30 分)

根据如下关系模式:

大学数据库关系模式如下:

department(dept_name, building, budget);

instructor(ID, name, dept_name, salary);

course(course_id, title, dept_name, credits);

Section, (course_id, sec_id, semester, year, building, room_number, time_slot_id);

teaches(ID, course_id, section_id, semester, year);

student(ID, name, dept_name, tot_cred);

classroom(building, room_number, capacity)

time_slot(time_slot_id, day, start_time, end_time)

使用关系代数和 SQL 语句完成如下查询操作:

1、查询在 2020 年秋天没有选课的学生的 ID.

$$\Pi_{ID}(\text{student}) - \Pi_{ID}(\delta_{\text{year} = 2020 \wedge \text{semester} = 'full'}(\text{takes}))$$

```
select ID from student
where ID not in (
    student T.ID
    from takes as T
    where year=2020 and semester="full")
```

2、查询选了名叫 Einstein 的老师开设的所有的课程的学生姓名。(老师和学生均无重名)

$$r_1 \leftarrow \Pi_{\text{course_id}}(\delta_{\text{name} = 'Einstein'}(\text{instructor} \bowtie \text{teaches}))$$

$$\Pi_{\text{name, course_id}}(\text{student} \bowtie \text{takes}) \div r_1$$

```

select S.name
from student as S
where not exist (
    select *
    from instructor as I
    nature join teaches as TE
    where I.name=' Einstein' and not exist (
        select *
        from takes as T
        where S.ID=T.ID and T.course_id=TE.course_id
    ))

```

3、查询在 2020 年秋季开课，但是不在 2021 年春季开课的所有课程 ID.

$$\pi_{\text{course_id}}(\sigma_{\text{semester} = \text{"fall"} \wedge \text{year} = 2020}(\text{Section})) - \pi_{\text{course_id}}(\sigma_{\text{semester} = \text{"spring"} \wedge \text{year} = 2021}(\text{Section}))$$

```

SELECT DISTINCT s.course_id

FROM Section s

WHERE s.semester = "fall" AND s.year = 2020

AND NOT EXISTS (

    SELECT 1

    FROM Section s2

    WHERE s2.course_id = s.course_id AND s2.semester = "spring" AND s2.year = 2021

);

```

4、给年薪大于 7 万元的老师涨5% 的工资，给年薪小于等于7万元的老师涨 10%的工资.

$$\text{instructor} \leftarrow \Pi_{\text{ID}, \text{name}, \text{dept_name}, \text{salary} * 1.05} (\delta_{\text{salary} > 70000} (\text{instructor})) \cup \Pi_{\text{ID}, \text{name}, \text{dept_name}, \text{salary} * 1.10} (\delta_{\text{salary} \leq 70000} (\text{instructor}))$$

```

UPDATE instructor
SET salary = salary * CASE

```

```

        WHEN salary > 70000 THEN 1.05
        WHEN salary <= 70000 THEN 1.10
        ELSE 1 -- 如果有不在这两种情况下的工资，可以保持不变，实际上可能不需要这个分支
    END;

```

5、不使用聚合函数，查询老师的最高工资。

$$\text{TopSalary} = \pi_{\text{salary}} \left(\sigma_{\text{salary} = (\pi_{\text{salary}}(\text{instructor}) - \pi_{\text{salary}}(\sigma_{\text{salary} < (\pi_{\text{salary}}(\text{instructor}))})}(\text{instructor})) \right)$$

这个表达式的意思是：

- 从 instructor 表中投影出 salary 列。
- 从这个投影中减去所有工资小于当前工资的行（这里使用了两次投影和一次差集操作）。
- 这将剩下具有最高工资的行。
- 最后，我们再次投影出这个结果集中的 salary 列，得到最高工资。

然后，使用这个最高工资值来选择 instructor 表中所有具有这个工资的老师：

$$\text{TopInstructors} = \sigma_{\text{salary} \in \text{TopSalary}}(\text{instructor})$$

这里如果用关系代数的思路写 SQL 会很麻烦，因此给出不一样的解法：（思路是先排序，再仅输出第一列）

```

select * from instructor
order by salary desc
limit 1

```


第二套试题（试卷部分）

一、简答题

- 1) 简述 varchar 与 char 的区别。
- 2) 什么是数据库索引?
- 3) 事务的四种特性指的是什么?
- 4) 在大学数据库中, 用 SQL 语句查询名字中包含'g' 学生的学号、姓名。
- 5) 简述函数和触发器之间的异同。
- 6) distinct 的作用是什么?
- 7) 若关系 R 所有的属性都是不可再分的数据项, 则该关系最低满足第几范式?
- 8) 简述用户自定义的类型和域之间的差别。
- 9) $AB \rightarrow C$ 能蕴含 $A \rightarrow C$, $B \rightarrow C$ 吗?

二、分析题

- 1、在学生表中, 将学号(ID)和姓名(nam)的组合设计为该表的主键是否合理?为什么
- 2、如果张三想通过汇款的方式转给李四 200 美元。张三的账户已经减掉 200 美元后系统发生故障, 并没有在李四账户中增加 200 美元, 请问数据库出现了什么样的状态?这个问题该怎么解决?数据库通过什么手段实现该操作?
- 3、请用阿姆斯特朗三定律(分解律、增强律和传递律)证明合并律。给出每一步骤的依据。

三、设计题

某汽车保险公司欲开发一套管理系统,, 其需要记录的数据如下:

顾客:顾客 ID、名字、家庭住址;

汽车:汽车编号、类型、所有顾客 ID;

事故:事故编号、日期、位置;

保险赔付:ID、截止日期、金额、实际赔付日期;

保险单:保险单号、保险赔付 ID;

其中, 每位顾客可以有 1 辆或多辆汽车; 每辆汽车可以关联 0 个或者多个事故, 每个事故可以关联 1 到多辆车; 1 个保险单可以覆盖 1 辆或多辆汽车, 1 辆车可以关联 1 到多个保险单; 保险单可关联一个或多个保险赔付。

完成如下设计:

- (1) 设计该信息管理系统的 E-R 图
- (2) 将该 E-R 图转换为关系模式;
- (3) 指出转换结果中每个关系模式的主码。

四、计算题

设有属于 1NF 的关系模式 $R=(A, B, C, D, E)$,

R 上的函数依赖集 $F=\{BC \rightarrow AD, AD \rightarrow EB, E \rightarrow C\}$ 。

- (1) R 是否属于 3NF?为什么?
- (2) R 是否属于 BCNF?为什么?

五、应用题

已知银行企业的数据库由以下表组成:

分行表 branch(branch-name, branch-city, assets)

客户表 customer(customer-name, customer-street, customer-city)

贷款明细表 loan(loan-number, branch-name, amount)

客户贷款表 borrower(customer-name, loan-number)

日存款明细表 account(account-number, branch-name, balance)

客户存款表 depositor(customer-name, account-number)

注:带下划线的属性为主码,假设客户的名字不相同

(1) 用 SQL 语句创建表。

(2) 使用关系代数和 SQL 语句找出在 Brighton 银行中有存款的所有客户的姓名存款号和存款额。

(3) 使用关系代数和 SQL 语句找出账户平均余额小于 5000 元的支行,显示支行名称及账户平均余额。

(4) 使用关系代数和 SQL 语句找出所有在银行中有贷款但无账户的客户

(5) 使用关系代数和 SQL 语句对所有存款余额大于平均存款额的账户付 3%的利息。

第二套试题参考答案:

一、简答题

1) 简述 varchar 与 char 的区别。

【解答】char 是一种固定长度的字符类型 varchar 则是一种可变长度的字符类型。

2) 什么是数据库索引?

【解答】索引是一种数据结构,是数据库管理系统中一个排序的数据结构,以协助快速查询、更新数据库表中数据

3) 事务的四种特性指的是什么?

【解答】原子性,一致性,隔离性,持久性

4) 在大学数据库中,用 SQL 语句查询名字中包含'g' 学生的学号、姓名。

【解答】Select ID, name from student where name like '%g%'

5) 简述函数和触发器之间的异同:

【解答】函数和触发器都是存储在数据库当中的一段代码。差别是函数需要显式调用,有返回值,触发器需要有触发事件,系统自动调用,无返回值。

6) distinct 的作用是什么?

【解答】删除查询结果中的重复记录

7) 若关系 R 所有的属性都是不可再分的数据项,则该关系最低满足第几范式?

【解答】第一范式

8) 简述用户自定义的类型和域之间的差别。

【解答】类型是强类型检查,无法定义约束;域是弱类型检查可以定义约束

9) $AB \rightarrow C$ 能蕴含 $A \rightarrow C$, $B \rightarrow C$ 吗?

【解答】根据阿姆斯特朗定理推理可知,不能。

二、分析题

1、在学生表中,将学号(ID)和姓名(name)的组合设计为该表的主键是否合理?为什么?

【解答】不合理。因为这种情况下,允许有多个姓名不同的学生对相同学号的情况。主键必须能够唯一确定一个元组。

2、如果张三想通过汇款的方式转给李四 200 元。张三的账户已经减掉 200 美元后系统发生故障，并没有在李四账户中增加 200 美元，请问数据库出现了什么样的状态?这个问题该怎么解决?数据库通过什么手段实现该操作?

【解答】数据库处于不一致的状态。应该进行事务回滚，即将张三账户中减掉的 200 元再加回来。利用数据库日志实现。

3、请用阿姆斯特朗三定律(分解律、增强律和传递律)证明合并律。给出每一步骤的依据。

【解答】

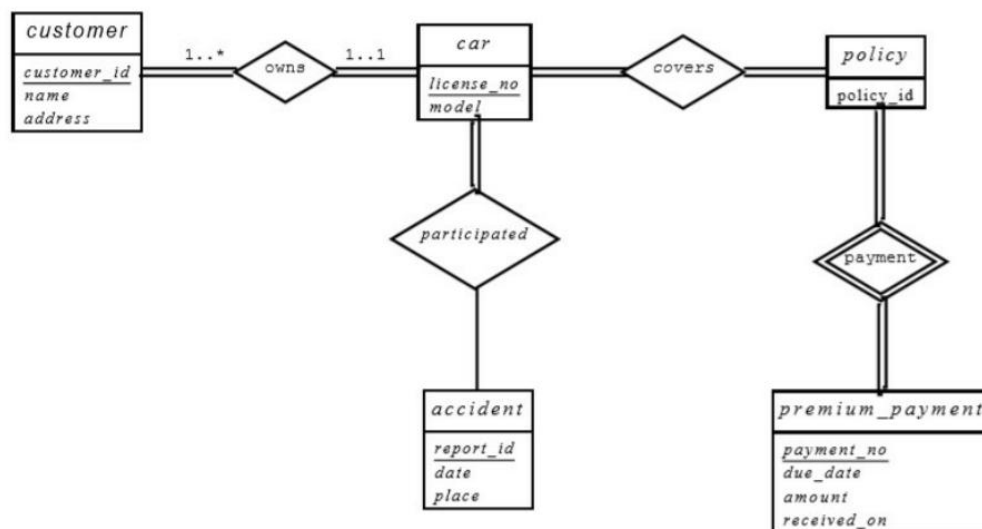
已知三定律，往证：若 $\alpha \rightarrow \beta, \alpha \rightarrow \gamma$ ，则 $\alpha \rightarrow \beta\gamma$ 。

证明：

$$\left. \begin{array}{l} \alpha \rightarrow \beta \xRightarrow{\text{增广律}} \alpha\gamma \rightarrow \beta\gamma \\ \alpha \rightarrow \gamma \xRightarrow{\text{增广律}} \alpha\alpha \rightarrow \alpha\gamma \text{ 即 } \alpha \rightarrow \alpha\gamma \end{array} \right\} \xRightarrow{\text{传递律}} \alpha \rightarrow \beta\gamma$$

三、设计题

【解答】



customer (customer_id, name, address)

car (license_no, model, customer_id)

policy (policy_id, payment_no)

accident (report_id, date, place)

participated (license_no, report_id)

premium_payment (payment_no, due_date, amount, received_no)

covers (license_no, payment_no, policy_id)

一、 计算题

设有属于 1NF 的关系模式 $R=(A, B, C, D, E)$,
R 上的函数依赖集 $F=\{BC \rightarrow AD, AD \rightarrow EB, E \rightarrow C\}$.

【解答】

- (1) 关系模式 R 的所有候选码为:BC, AD, BE。
- (2) 多函数依赖集 F 中, 所有的右部属性都在某个候选码的属性当中, 显然是 3NF
- (3) $E \rightarrow C$ 是非平凡依赖, 而 E 不是 R 的一个超码, 因此 R 不属于 BCNF

五、应用题

- (3) 用 SQL 语句创建表。

```
create table branch(  
branch-name char(15) primary key  
branch-city char(30)  
assets numeric(16,2)  
primary key(branch-name));
```

- (4) 使用关系代数和 SQL 语句找出在 Brighton 银行中有存款的所有客户的姓名、存款号、和存款额。

$\Pi_{customer-name, depositor.account-number, balance} (\sigma_{depositor.account-number=account.account-number \wedge branch-name='Brighton'} (depositor \times account))$

Sql 语句:

```
select customer-name, depositor.account-number, balance  
from depositor, account  
where depositor.account-number=account.account-number and  
and branch-name='Brighton' ;
```

- 3) 使用关系代数和 SQL 语句找出账户平均余额小于 5000 元的支行, 显示支行名称及账户平均余额

$\sigma_{ab < 5000} (\rho_{branch-balance(branch-name, ab)} (branch-name \mathcal{G}_{avg(balance)} (account)))$

```
select branch-name, avg(balance)  
from account  
group by branch-name  
having avg(balance) < 5000
```

- (4) 使用关系代数和 SQL 语句找出所有在银行中有贷款但无账户的客户。

$\prod_{customer-name} (borrower) - \prod_{customer-name} (depositor)$

```
select distinct customer-name  
from borrower  
where customer-name not in  
(select customer-name from depositor)
```

(5) 使用关系代数和 SQL 语句对所有存款余额大于平均存款额的账户增加 3% 的利息。

$$account \leftarrow \Pi_{account-number, branch-name, balance * 1.03} (\sigma_{balance > avg(balance)}(account)) \\ \cup \Pi_{account-number, branch-name, balance} (\sigma_{balance \leq avg(balance)}(account))$$

```
update account
set balance=balance*1.03
where balance>(select avg(balance) from account)
```

第三套数据库考试试题

考试时间:2018 年 6 月 25 日

一、简答题(每题 2 分, 共 20 分)

1、为什么要为数据库加严格两阶段锁或强两阶段锁?

【解答】不仅可以确保冲突可串行化, 还避免了级联回滚, 提高了效率和安全性。

2、为什么要在数据库中引入事务的概念?

【解答】事务的 ACID 特性很契合数据库的操作, 为数据库提供了一种从故障到恢复正常的方法, 可以有效地保持数据库的一致性, 提高数据库系统的效率和安全。

3、为什么要在数据库中设置多种不同粒度的锁?

【解答】主要用于并发控制, 提高性能, 避免死锁, 满足不同的操作需求。通过允许更细粒度的并发, 可以减少锁的争用, 从而提高数据库操作的吞吐量, 不同粒度的锁可以更精确地控制对数据的访问, 减少不必要的锁定, 从而更有效地利用数据库资源。

4、为什么要对数据库的调度就行可串行化判别, 其实际意义是什么?

【解答】可串行化是并发事务正确调度的准则, 只有判别出调度符合该准则, 才能知道是正确的调度。

5、若对数据库进行 BCNF 范式的分解, 从而导致没有保持依赖, 在数据库设计中要如何解决?

【解答】可以改为采用 3NF 分解; 增加冗余属性。

6、数据库中, 实体的完整性是如何被保证的?

【解答】设置主键, 保证能够唯一的标识对应的实体。

7、如何降低数据库中数据的冗余度?

【解答】采用无损分解下的 BCNF 分解、3NF 分解。

8、如何标识一个弱实体集?

【解答】标识实体集的主键+自身的分辨符

9、在 SQL 语句中, 如何表示除法运算?

【解答】使用如下结构语句: `not exist (.....not exist(.....) )`

10、关系代数中, 与等值连接相比, 自然连接的缺点是什么?

【解答】自然连接的连接条件无法自定义, 缺少灵活性, 难以处理不同的数据类型。

二、分析题(每题 5 分, 共 20 分)

1、证明如下结论:

如果关系 $r(R)$ 和 $s(S)$ 不含有任何相同属性, 即 $R \cap S = \Phi$, 那么 $r \bowtie s = r \times s$ 。

【解答】

自然连接的定义

自然连接 $r \bowtie s$ 是基于两个关系共有属性的等值连接。数学上, 如果 $R \cap S \neq \emptyset$, 自然连接可以定义为:

$$r \bowtie s = \pi_R(r) \bowtie_{\theta} \pi_S(s)$$

其中, π_R 和 π_S 分别是 r 和 s 中投影出属性集合 R 和 S 的操作, θ 是基于共有属性的等值条件。

笛卡尔积的定义

笛卡尔积 $r \times s$ 可以定义为:

$$r \times s = \{(t_1, t_2) \mid t_1 \in r, t_2 \in s\}$$

这里 (t_1, t_2) 是一个有序对, 其中 t_1 是 r 中的一个元组, t_2 是 s 中的一个元组。

证明

由于 $R \cap S = \emptyset$, 我们有 $\pi_R(r) = r$ 和 $\pi_S(s) = s$, 因为没有属性被投影掉。此时, 自然连接的定义变为:

$$r \bowtie s = r \bowtie_{\theta} s$$

但是, 由于没有共有属性, θ 无法定义 (因为没有属性可以用于等值比较), 因此自然连接退化为笛卡尔积: $\square$

$$r \bowtie s = \{(t_1, t_2) \mid t_1 \in r, t_2 \in s\}$$

这正是笛卡尔积的定义。因此, 我们得到:

$$r \bowtie s = r \times s$$

2、某人在使用移动支付系统时, 输入了正确的支付密码并提交后, 发生了断网情况, 导致支付失败, 分析该事务经历了哪几个状态?

【解答】活动的: 事务开始, 用户输入支付信息并提交支付请求。

部分提交的: 如果支付系统的数据库在断网前已经执行了事务的最后一条语句, 但因为网络问题无法完成后续的提交确认步骤, 事务可能处于部分提交状态。

失败的: 如果系统检测到因为断网导致事务无法继续执行, 事务将进入失败状态。

中止的: 在事务失败后, 系统将执行回滚操作, 撤销所有已经进行的操作, 确保数据库状态回到事务开始之前。一旦回滚完成, 事务将变为中止状态。

3、某学校教务管理系统为每个学院提供了固定的上课教室, 系统规定不同学院的教务办负责人员登录到本系统后, 只能有权支配各自学院的教室资源, 请问需要定义哪种数据库对象实现此需求? 为什么?

【解答】定义视图, 用以展示各自学院的资源, 该视图只展示该学院的教室资源。教务办负责人员只能通过这些视图访问数据; 定义角色, 并为他们赋予相应的权限。

4、某学校教务管理系统有学生表“(学号, 姓名, 出生日期, 学院编号)”和“选课信息表(学号, 课程编号, 成绩)”。现有如下规定: 如果某学生退学的话, 则自动删除该生的所有选课记录: 该如何定义此业务逻辑?

【解答】要定义这种业务逻辑, 通常需要在数据库层面使用外键约束和级联规则, 而这里使用触发器会导致工作量很大、效率不好。

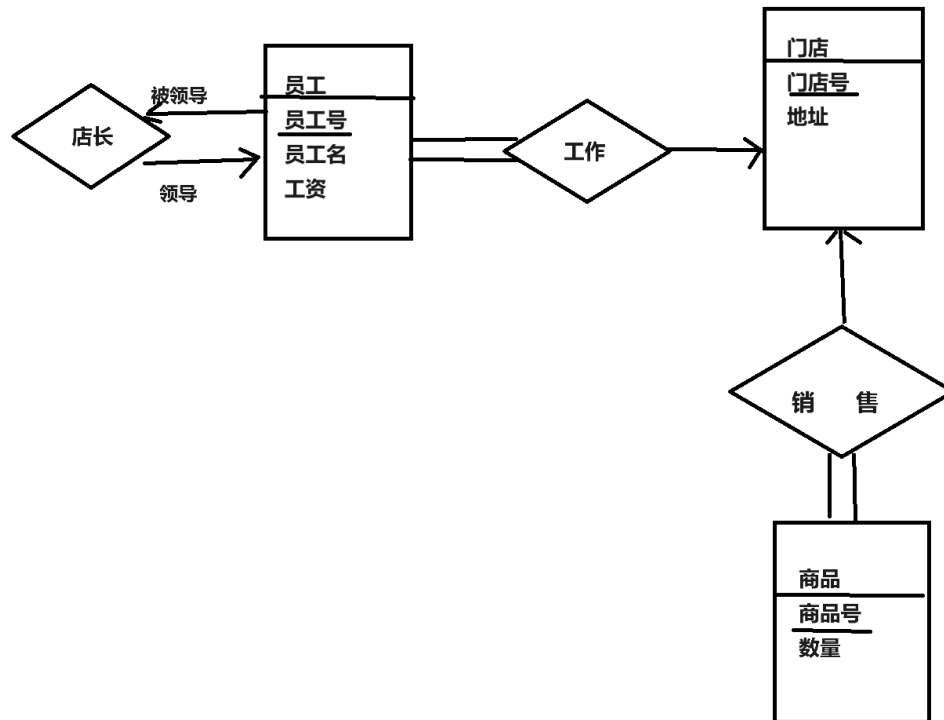
三、设计题(15 分)

某连锁超市有若干门店，若干员工和若干商品。每个门店有若干员工和一个店长，店长负责管理本门店的所有员工，每个员工只在特定门店工作。各个门店销售的商品基本相同，由于销售差异。每个门店所拥有的特定商品的数量不同。

1、根据上述需求，画出 E-R 模型；

2、将 E-R 模型转化为相应关系模型，并说明每个关系模式中必要的完整性约束。

【解答】1.



2.

员工 (员工号, 员工名, 工资) 主码为员工号

门店 (门店号, 地址) 主码为门店号

商品 (商品号, 数量) 主码为商品号

销售 (商品号, 门店号)

工作 (员工号, 门店号)

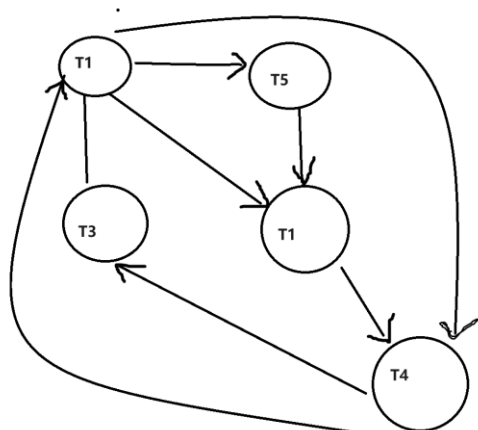
主码为下滑线部分。

四、计算题(15 分)

1、设有如图所示调度 s , 判别 S 是否为冲突可串行化调度?如果是, 则给出与 s 等价的一个串行调度:如果不是, 说明为什么。

时序	T1	T2	T3	T4	T5
1	R(A)				
2		W(B)			
3			R(A)		
4				W(C)	
5					R(B)
6	W(B)				
7		R(A)			
8		R(C)			
9			W(C)		
10				R(B)	

【解答】不是冲突可串行化调度, 因为它的优先图有环路。如下:



2、设有关系模式 $R=(A, B, C, D, E, G, H)$, 函数依赖 $F=\{AH \rightarrow C, C \rightarrow A, CH \rightarrow D, C \rightarrow EG, EH \rightarrow C, CG \rightarrow DH, CE \rightarrow AG, ACD \rightarrow H\}$

- (1) 令 $X=BC$, 求 X 关于 F 的闭包 X^+ ;
- (2) 将关系 R 逐步分解为 3NF

【解答】

1. 容易推出, BC 是 R 的候选码, 因此 $X^+=ABCDEGH$;
2. 先求出 F 的正则覆盖为 $F_c = \{F \rightarrow GH, H \rightarrow FI, A \rightarrow BC, AF \rightarrow DE\}$

再求候选码:

在原 F 中只在左边出现的: A

只在右边出现的: $GJBCDE$

两边都出现: F, H . 然后计算可知:

$$A^+ = ABC$$

$$(AF)^+ = (AH)^+ = R$$

$$(EH)^+ = R$$

因此，候选码为 BC, AH, EH。

由此分解得：R1=FGH, R2=ABC, R3=ADEF, R4=FHI

五、应用题(20 分)

有一个手机游戏平台，有若干游戏供玩家下载，该应用抽象成以下关系模式：

GAME(Gid, Gname, version, type)：

对应中文为：游戏 id，游戏名版本号，游戏类别

PERSON(Pid, Pname, age, hobby)；

对应中文为：玩家 id，玩家昵称，年龄，爱好

PG(Pid, Gid)；

1、分别用关系代数和 SQL 语句查询：玩家“wxy”下载的所有游戏的名称：

$$\pi_{Gname}(GAME \bowtie_{PG.Pid=PERSON.Pid} PERSON \bowtie_{\sigma_{Pname='wxy'}}(PERSON))$$

```
SELECT G. Gname
```

```
FROM GAME G, PERSON P, PG
```

```
WHERE G. Gid = PG. Gid AND P. Pid = PG. Pid AND P. Pname = 'wxy';
```

2、分别用关系代数和 SQL 语句查询，下载数量超过 500 次的游戏名称：

$$\pi_{Gname}(\sigma_{|PG|>500}(PG \times GAME))$$

```
SELECT G. Gname
```

```
FROM GAME G
```

```
WHERE (
```

```
    SELECT COUNT(*)
```

```
    FROM PG
```

```
    WHERE PG. Gid = G. Gid
```

```
) > 500;
```

3、分别用关系代数和 SQL 语句查询：下载了游戏'G01' 没下载游戏'G02' 的玩家姓名；

$$\pi_{Pname}(\sigma_{Pid \in (PG \bowtie_{\sigma_{Gid='G01'}}(GAME)) - (PG \bowtie_{\sigma_{Gid='G02'}}(GAME))}(PERSON))$$

```
SELECT DISTINCT P. Pname
```

```
FROM PERSON P
```

```
WHERE EXISTS (
```

```
    SELECT 1
```

```
    FROM PG
```

```
    WHERE PG. Pid = P. Pid AND PG. Gid = 'G01'
```

```
)
```

```
AND NOT EXISTS (
```

```
    SELECT 1
```

```

FROM PG
WHERE PG.Pid = P.Pid AND PG.Gid = 'G02'
);

```

4、用关系代数完成查询:下载了所有“益智类”游戏的玩家 id;

$$\pi_{Pid}(PG \bowtie \sigma_{type='益智类'}(GAME))$$

5、用 SQL 语句完成查询:查询年龄 15-25 岁之间的玩家都下载了哪些游戏;

```

SELECT DISTINCT G.Gname
FROM GAME G
JOIN PG ON G.Gid = PG.Gid
JOIN PERSON P ON PG.Pid = P.Pid
WHERE P.age BETWEEN 15 AND 25;

```

6、用户 'p01' 下载了游戏“G02”，在数据库系统中应当发布的 SQL 语句是什么？

```

INSERT INTO PG (Pid, Gid) VALUES ('p01', 'G02');

```

7、用 SQL 语句完成;游戏开发者修改了游戏后重新上传,版本号发生了变化,数据库完成什么操作。

```

UPDATE GAME
SET version = '新版本号'
WHERE Gid = '受影响的游戏 ID';

```

第四套数据库试题

一、简答题

1. 系统开发人员通过层次抽象来对用户屏蔽复杂性，请根据层次由低到高列出各个层次，并说明 DBA（数据库管理员）主要使用哪个层次抽象。

答：物理层、逻辑层、视图层。DBA 主要使用逻辑层抽象。

2. 解释一下物理数据独立性，说明数据库语言的分类及其基本用途，并说出哪种类型语言输出放在数据字典中，简述空值（null）表示的含义。

答：物理数据独立性：应用程序如果不依赖于物理模式，它们就被称为是具有物理数据独立性，因此即使物理模式改变了，它们也无须重写。数据库语言的分类及用途：数据定义语言（DDL），用于定义数据库模式；数据操纵语言（DML），用于表达数据库的查询和更新。DDL 的输出放在数据字典中。空值表示这个值不存在或者未知，未知值可能是缺失或不知道。

二、关系代数

设有三个关系：

S (S#, SNAME, AGE, SEX)

SC (S#, C#, CNAME)

C (CH, CNAME, TEACHERNAME)

试用关系代数表达式表示下列查询语句：

（注：S 代表学生，SC 代表学生-课程关系，C 代表课程）

(1) 检索 LIU 老师所授课程的课程号和课程名

【解答】 $\Pi_{CH, CNAME}(\sigma_{TEACHERNAME = "LIU"}(C))$

(2) 检索年龄大于 19 岁的男学生的学号和姓名（性别男用 "M" 表示，女用 "F" 表示）。

【解答】 $\Pi_{S\#, SNAME}(\sigma_{AGE > 19 \wedge SEX = "M"}(S))$

(3) 检索学号为 "S3" 的学生所学课程的课程名与任课教师姓名。

【解答】 $\Pi_{CNAME, TEACHERNAME}(\sigma_{S\# = "S3"}(SC \bowtie C))$

(4) 检索 WANG 同学不学的课程的课程号。

【解答】 $\Pi_{C\#}(C) - \Pi_{CH}(\sigma_{SNAME = "WANG"}(S \bowtie SC))$

(5) 检索选修课程包含 LU 老师所授全部课程的学生学号。

【解答】 $\Pi_{S\#, C\#}(SC) \div \Pi_{C\#}(\sigma_{TEACHERNAME = "LU"}(C))$

三、问答

我们知道，SQL 是基于关系代数的，那么对于 SQL 表达式基本结构的三个子句 select, from 和 where. SQL 的构造方式及顺序是怎样的？

【解答】SQL 先构造 from 子句中关系的笛卡尔积，根据 where 子句中的谓词进行关系代

数的选择运算，然后将结果投影到 `select` 子句中的属性上。

四、SQL 语句

考虑下图所示的员工数据库，为下面每个查询写出 SQL 表达式：

`employee (employee_name, street, city)`

`works (employee_name, company_name, salary)`

`company (company_name, city)`

(1) 找出所有为 Microsoft 工作的员工的姓名。

```
Select employee_name
From works
Where company_name='Microsoft'
```

(2) 修改数据库存储的信息，使得 Mike 现在居住在 London.

```
Update employee
Set city='London'
Where employee_name='Mike'
```

(5) 找出各个公司员工的平均薪水，并按照公司名称逆序排序

```
Select company_name, avg(salary)
From worksGroup by company_name
Order by company_name desc
```

(6) 为 Microsoft 所有员工增加 15%的薪水，

```
Update works
Set salary=salary*1.15
Where company_name='Microsoft'
```

二、应用

假设有下面的两个关系模式:职工(职工号, 姓名, 年龄, 职务, 工资, 部门号), 其中职工号为主码;部门(部门号, 名称, 经理名, 电话), 其中部门号为主码。用 SQL 语言定义这两个关系模式,要求在模式中完成以下完整性约束条件的定义:定义每个模式的主码;定义参照完整性;定义职工年龄不得超过 54 岁。

```
create table dept
(deptno number(2),
deptname varchar(10) ,
manager varchar(10),
phonenum char(12),
constraint PK1 primary key (deptno));
create table emp
(empno number(4),
empname varchar(10),
age number(2),
```

```

constraint C1 check (age<=54),
job varchar(10)
salary number(7,2)
deptno number(2),
constraint PK2 primary key (empno),
constraint FK _DEPTNO foreign key (deptno) references dept(deptno) );

```

六、计算题

1. 已知如下关系模式: $R=(A, B, C)$, $S=(D, E, F)$, 设关系 $r(R)$ 和 $s(S)$ 已知, 分别给出与下列表达式等价的元组关系演算表达式:

(1) $\Pi_A(r)$

【解答】 $\{t \mid \exists q \in r(q[A] = t[A])\}$

(2) $\Pi_{A,F}(\sigma_{C=D}(r \times s))$

【解答】 $\{t \mid \exists p \in r, \exists q \in s(t[A] = p[A] \wedge t[F] = q[F] \wedge p[C] = q[D])\}$

2. 设 $R=(A, B, C)$, 且 r_1 和 r_2 都是模式 R 上的关系。分别给出与下列表达式等价的域关系演算表达式:

(1) $\sigma_{B=17}(r_1)$

【解答】 $\{ \langle a, b, c \rangle \mid \langle a, b, c \rangle \in r_1 \wedge b = 17 \}$

(2) $\Pi_{A,B}(r_1) \bowtie \Pi_{B,C}(r_2)$

【解答】 $\{ \langle a, b, c \rangle \mid \exists p, q (\langle a, b, p \rangle \in r_1 \wedge \langle q, b, c \rangle \in r_2) \}$

七、请比较关系代数与 Datalog。(两点即可)

【解答】(1) 基本关系代数的每一种操作都可以用 Datalog 查询表达, 但扩展关系代数中的分组和聚集等操作无法用 Datalog 表达; (2) 任何单个 Datalog 规则都可以用关系代数表达, 但 Datalog 可以表示递归, 而关系代数不能。

八、根据自然语言描述绘制某研究院的 E-R 图:

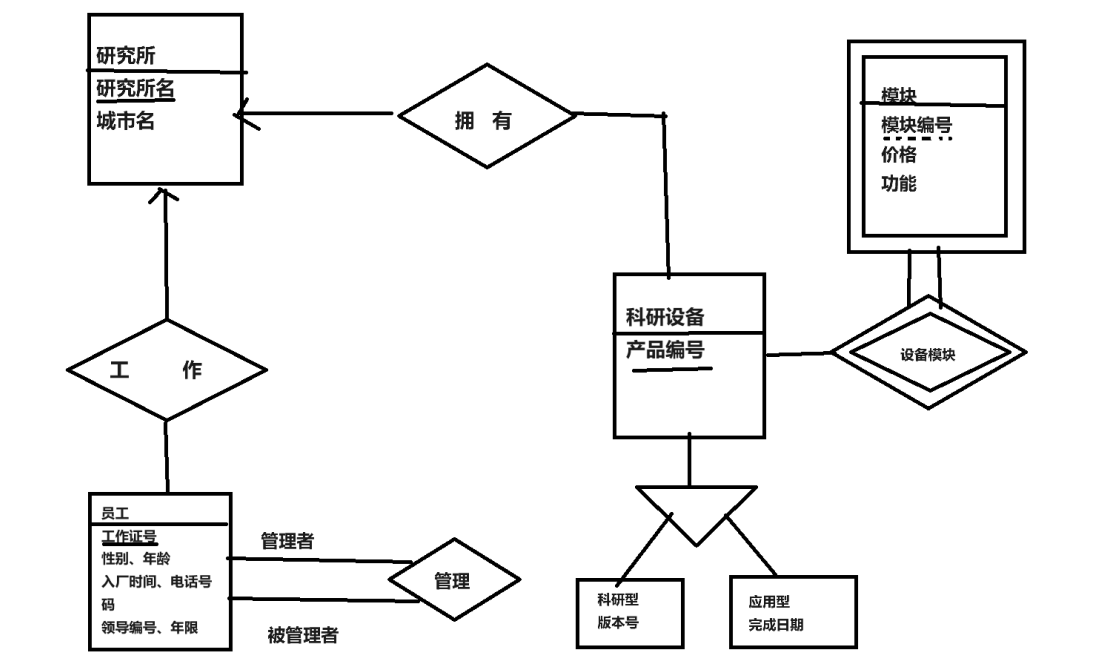
(1) 研究院由多个研究所组成, 每个研究所由唯一的名字标识, 位于某个城市, 研究院管理每个研究所的资产。

(2) 研究院的员工由其工作证号来标识, 还需要记录每个员工的姓名、电话号码(包括办公电话和手机电话)及其直属领导的工作证号, 研究院还需要知道员工入厂的日期, 由此日期可以推知员工的工作年限。

(3) 研究所开发的产品各不相同, 分别赋予不同的产品编号。每种产品均分为科研型和应用型两种, 科研型产品应记录版本号, 应用型产品应记录该产品的完成日期。

(4) 每个产品由若干个模块构成, 每个模块用模块号标识。研究院需要记录每个模块的具体功能和价格, 虽然不同产品的模块号可能重复, 但同一产品的模块号是不重复的。

【解答】



九、解释全部的和部分的这两种设计约束之间的差别，说说你的理解。

【解答】对于一个全部的设计约束，每个高层实体必须属于一个低层实体集，部分的设计约束则允许一些高层实体不属于任何低层实体集。举例来说，工作组实体集就是部分化的例子，由于员工在工作三个月后才分配到某个具体工作组中，某些 employee 实体可以不属于任何具体低层工作组实体集。

十、根据所学知识，说明属性集闭包的算法的用途。

【解答】①判定 α 是否为超码，通过计算 α^+ ，看 α^+ 是否包含了 R 中的所有属性

②通过检验是否 β 含于 α^+ ，可以验证函数依赖 $\alpha \rightarrow \beta$ 是否成立，即是否在 F^+ 里。也就是说，用属性闭包计算 α^+ ，看它是否包含 β 。

③该算法给了另一种计算 F^+ 的方法：对任意的 r 含于 R ，找出闭包 r^+ ；对任意的 S 包含于 r ，输出一个函数依赖 $r \rightarrow S$ 。

十一、函数依赖计算题

设有属于 1NF 的关系模式 $R=(A, B, C, D, E)$ ， R 上的函数依赖集 $F=\{BC \rightarrow AD, AD \rightarrow EB, E \rightarrow C\}$ 。

1. 计算 $(BC)^+$ 。

【解答】 $(BC)^+=ABCDE$

2. R 是否属于 3NF？为什么？

【解答】是 3NF。因为 R 的候选码为 BC, AD ；对于非平凡函数依赖 $BC \rightarrow AD$ 和 $AD \rightarrow EB$ ，满足函数依赖的左部为候选码；而对于非平凡函数依赖 $E \rightarrow C$ ，满足函数依赖的右部为候选码中的属性。

3. R 是否属于 BCNF?为什么?

【解答】不满足 BCNF, 因为非平凡函数依赖 $E \rightarrow C$, E 不是 R 的候选码。

十二、试述实现数据库安全性控制的常用方法和技术。(三点即可)

【解答】①用户标识和鉴别:该方法内系统提供一定的方式让用户标识自己的名字或身份。每次用户要求进入系统时,由系统进行核对,通过鉴定后才提供系统的使用权。

②存取控制:通过用户权限定义和合法权检查确保只有合法权限的用户访问数据库,所有未被授权的用户无法存取数据。

③视图机制:为不同的用户定义视图,通过视图机制把需要保密的数据对无权存取的用户隐藏起来,从而自动地对数据提供一定程度的安全保护。

④审计追踪:建立审计日志,把用户对数据库的所有操作记录下来放入审计日志中,DBA 可以利用审计跟踪的信息,重现导致数据库现有状况的一系列事件,找出非法存取数据的人、时间和内容等。从而使得不知道解密算法⑤数据加密:对存储和传输的数据进行加密处理,的人无法获知数据的内容。

十三、请写出一个 SQL 触发器来执行下列动作:

假设插入元组的电话号码域的值为空时,表明没有电话号码,定义一个名为 setnull_trigger 的触发器,将值用 null 代替(值为空即为 phonenumber=" ",关系名用 r 代替,电话号码域用 phonenumber 标识)。

【解答】

```
Create trigger setnull_trigger before update on r
Referencing new row as nrow
For each row
When nrow.phonenumber=" "
Set nrow.phonenumber=null
```

十四、比较静态散列与动态散列技术。

【解答】静态散列所用散列函数的桶地址集合是固定的,这样的散列函数不容易适应数据库随时间的显著增加。有几种允许修改散列函数的动态散列技术,可扩充散列是其中之一,它可以在数据库增长或缩减时通过分裂或合并桶来应付数据库大小的变化。

十五、在同一关系上能否对两个不同查找键值建立两个主索引?为什么?

【解答】在同一关系上不允许对两个不同查找键值建立两个主索引,这是因为大多数情况下,主文件的顺序不可能同时与两个索引的查找键值顺序一致。

十六、回答问题

1. 根据数据库查询处理和查询优化理论,说出执行计算包含多个运算的表达式的两种方法。

【解答】实体化、流水线(管道线)。

大致的步骤可以归纳如下:

② 把查询转换成某种内部表达,通常用的内部表达是语法树。

②把语法树转换成标准(优化)形式,即利用优化算法,把原始的语法树转换成优化的形式。

③选择低层的存取路径。

④生成查询计划,选择

2. 试述查询优化的一般步骤。

【解答】其中代价最小的一个。

3. 判断正误:在生成等价关系代数表达式时,不会生成所有的等价的关系代数表达式,只会生成有限的。

【解答】正确。

第五套数据库试题

2015 年

一、简答题

1. 数据库中常用的完整性约束包括哪些?

【解答】主码约束, 非空约束, 唯一性约束, check 子句, 外键约束

2. 简述数据库系统与文件系统的主要区别。

【解答】文件系统: 以文件和文件夹的形式组织和存储数据, 缺少事务支持、并发控制、完整性约束, 一致性差。数据库系统: 使用结构化的数据模型组织和存储数据, 具有事务支持、并发控制、完整性约束, 一致性。

3. 一般情况下, 关系 R 与关系 S 要进行自然连接, 需要满足什么条件?

【解答】R 与 S 要有属性域的一致性: 参与连接的属性在两个关系中必须具有相同的数据类型, 以便可以进行比较。

4. 为什么要在数据库中引入事务的概念?

【解答】在数据库中引入事务的概念主要是为了确保数据的完整性和可靠性, 支持高并发的数据处理需求, 同时事务为数据提供了一种从故障到恢复正常的办法。

5. 已知视图 faculty 的定义: `create view faculty as select id, name, dept name from instructor`; 当发布 `insert into faculty values ('30765', 'Green', 'Music')` 命令时, 数据库系统如何执行?

【解答】检查插入约束是否满足视图的可更新性, 将数据从插入到表 `instructor` 中, 视图为存储的 `select` 语句, 也会随之自动更新。

6. 用基本关系代数表达式来表示 R 交 S。

【解答】 $R \cap S = R - (R - S)$

7. 某银行为不同储蓄金额执行不同利率: 低于 10 万元的一年支付 4.2% 的利息, 10 万元以上 (包括 10 万) 一年支付 4.5% 的利息。假设储蓄账户表 `account (aid, balance, branch_name)`, 写出与上述业务相对应的 SQL 语句, 如何保证其执行结果的正确性?

【解答】SELECT

aid,

balance,

branch_name,

CASE

WHEN balance < 100000 THEN balance * 0.042

ELSE balance * 0.045

END AS interest

FROM

account;

需要考虑的点: 完整性约束: 确保 `account` 表有适当的完整性约束, 如 `balance` 列的非负约束。事务管理: 如果计算利息是作为事务的一部分, 确保使用事务来保证计算和更新过程的原子性, 避免两次计算利息。

8. 什么样的关系模式满足 BCNF 范式?

【解答】BC 范式：

F^+ 中的任何一个 $\alpha \rightarrow \beta$ 要么是平凡的，要么 α 是超码。只有满足上述条件，该关系模式 R 才满足 BCNF 范式。

9. 关系数据库中，超码、候选码、主码有什么区别？

【解答】候选码为一个或多个属性组成的整体，能够唯一标识一个元组。候选码是超码的子集，超码是候选码的超集，主码为多个候选码中的任意一个。

10. 关系代数中，选择操作的功能是什么？

【解答】选择出关系中满足条件的元组。

11. 简述数据库中引入封锁机制的优缺点

【解答】优点：可以确保调度的可串行化的良好执行，某些机制还可以避免级联回滚；缺点：可能发生死锁、饥饿等情况，影响效率。

12. 数据库中，调度指的是什么？

【解答】调度指的是指令在系统中依照某种顺序执行。

13. 事务由哪几个状态组成？判断当手机支付已提交，但由于网络信号消失而导致支付失败，此时，事务处于何种状态？

【解答】事务的状态有：活动的：初始状态，事务执行时处于这个状态；部分提交的：最后一条语句执行后；失败的：发现正常的执行不能继续后；中止的：事务回滚并且数据库已恢复到事务开始执行前的状态后；提交的：成功完成后。

手机支付可能经过以下状态：

1. **活动的 (Active)**：事务开始，用户输入支付信息并提交支付请求。
2. **部分提交的 (Partially Committed)**：如果支付系统的数据库在断网前已经执行了事务的最后一条语句，但因为网络问题无法完成后续的提交确认步骤，事务可能处于部分提交状态。
3. **失败的 (Failed)**：如果系统检测到因为断网导致事务无法继续执行，事务将进入失败状态。
4. **中止的 (Aborted)**：在事务失败后，系统将执行回滚操作，撤销所有已经进行的操作，确保数据库状态回到事务开始之前。一旦回滚完成，事务将变为中止状态。

14. 在 MySQL 中定义 foreign key 时，与标准 SQL 有何区别？

【解答】MySQL 不支持外键引用时的缺省情况，而要求指定被应用关系的确切属性。

15. MySQL 中不支持 INTERSECT 语句，可以用哪些语句来代替？

【解答】可以用多个 SELECT 语句与 AND 逻辑结合：not exists、join 等子句代替

16. 数据库中如何判断死锁是否发生？

【解答】用等待图来判别，当且仅当产生环路时发生死锁。

17. 什么样的调度一定能够保证数据的一致性？

【解答】可串行化调度。

18. 为了保证数据库的并发性，引入了哪些锁？

【解答】共享锁，排它锁；显式锁，隐式锁。

19. 为什么事务非正常结束时会影响数据库数据的正确性，请举例说明。

【解答】事务的原子性被破坏，导致数据的一致性破坏。

假设一个银行系统正在处理一笔转账操作，该操作涉及两个事务：

事务 1：从账户 A 扣除 100 元。

事务 2：向账户 B 存入 100 元。

这两个事务需要作为一个整体来执行，以保证转账操作的原子性。如果事务 1 成功执行（账户 A 的余额减少 100 元），但事务 2 因为某种原因（如系统崩溃、网络问题等）未能执行，那么数据库数据将出现以下问题：

数据不一致：账户 A 的余额已经减少，但账户 B 的余额没有增加，导致银行的总资产减少 100 元，违反了事务前的一致状态。

违反原子性：转账操作作为一个整体未能成功执行，因为只有部分操作（事务 1）完成了。

违反一致性：数据库从一个一致的状态转移到了另一个不一致的状态，因为账户 A 的扣款没有对应的账户 B 的存款。

20, 利用 Armstrong 公理证明伪传递率: 若有 $A \rightarrow B, CB \rightarrow D$, 则有 $AC \rightarrow D$ 。

【解答】

$$\left. \begin{array}{l} A \rightarrow B \xRightarrow{\text{增广律}} AC \rightarrow CB \\ CB \rightarrow D \end{array} \right\} \xRightarrow{\text{传递律}} AC \rightarrow D \quad \text{即证}$$

二、分析题(每题 5 分, 共 20 分)

1. 两个人分别在去哪儿网和携程网上购买 2017 年 7 月 2 日, CZ6147 次航班, 从长春飞往北京, 但该航班的经济舱只剩一张票, 两个人同时下单, 数据库中要如何控制?

【解答】数据库对两个事务的调度要确保 ACID 特性, 两人同时下单, 但是数据库收到信息有先后顺序, 因此数据库应当为先提交者服务, 组织后提交者的事务并反馈。

2. 很多在线手机游戏都支持离线操作, 即当网络不通时, 可以离线玩, 等联网之后再进行数据同步在这个过程中, 可能涉及数据库的哪些概念?

【解答】事务管理、ACID 特性、日志管理、数据库恢复。

3. 用户到银行开了一个储蓄账户(account 表), 可以随时对该账户进行存钱、取钱等管理, 分别对应数据库中的什么操作? 在 ATM 机上取钱, 当卡内余额不足时, ATM 机不做任何支付, 如何将此规则定义在该表上?

【解答】用户到银行开立储蓄账户并进行管理, 对应的数据库操作通常包括:

存钱: INSERT 或 UPDATE 操作, 增加账户余额。

取钱: UPDATE 操作, 减少账户余额。

在 ATM 机上取钱时, 如果卡内余额不足, 可以通过以下方式定义规则: 使用 CHECK 约束来保证账户余额充足。应用事务并结合适当的异常处理逻辑, 确保在余额不足时回滚事务。

4. 吉林大学一卡通实现了与银行的绑定, 当一卡通卡内余额小于某一设定金额的时候, 可以通过银行卡直接转账, 请问如何实现自动转账? 转账的金额在数据库端如何设定?

【解答】要实现自动转账的功能, 可以通过设置触发器来完成。触发器是一种特殊的数据库对象, 它会在满足特定条件时自动执行。或者采用外键约束和级联技术。

三、设计题(共 15 分)

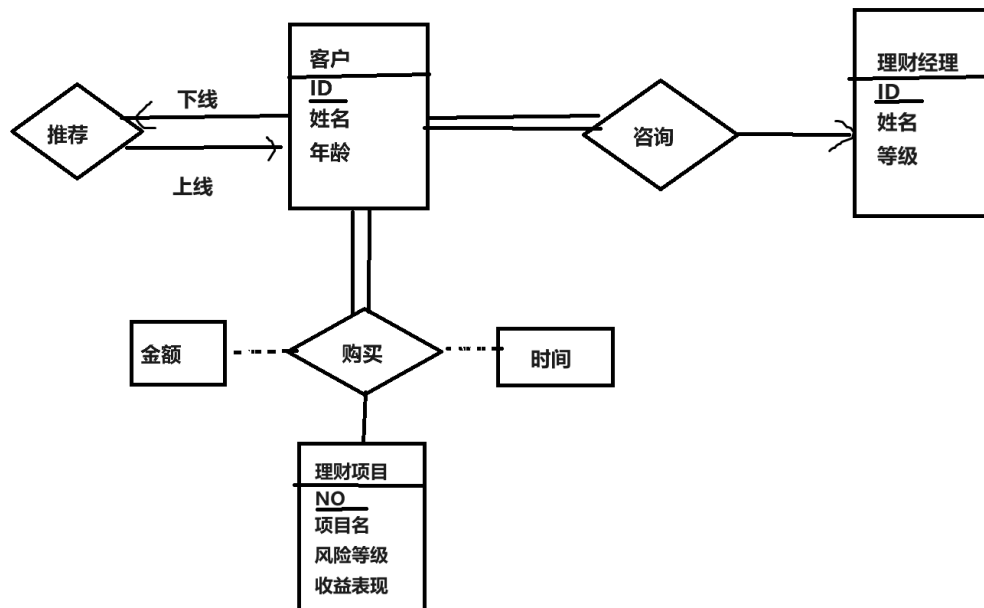
某金融公司为客户提供理财服务。公司提供股票, 基金, 外币等多种投资理财项目, 并且为每位客户配备一个理财经理, 为客户提供投资咨询服务, 客户可以购买一种或多种理财产品,

根据买卖的时间和购买的金额来计算收益。每个客户还可以推荐下线客户，每个客户只能有一个上线。

1. 根据上述需求，画出 E-R 模型：(7 分)

2. 将 E-R 模型转化为对应的关系模型，并说明每个关系模式中需要设置的完整性约束。(8 分) 注：实体的属性根据实际情况自行设定，每个实体不少于三个属性，不多于五个属性。

【解答】1.



2.

客户（客户 ID, 姓名，年龄，上线客户 ID，理财经理 ID）

理财经理（理财经理 ID, 姓名，等级）

购买（客户 ID, NO, 金额，时间）

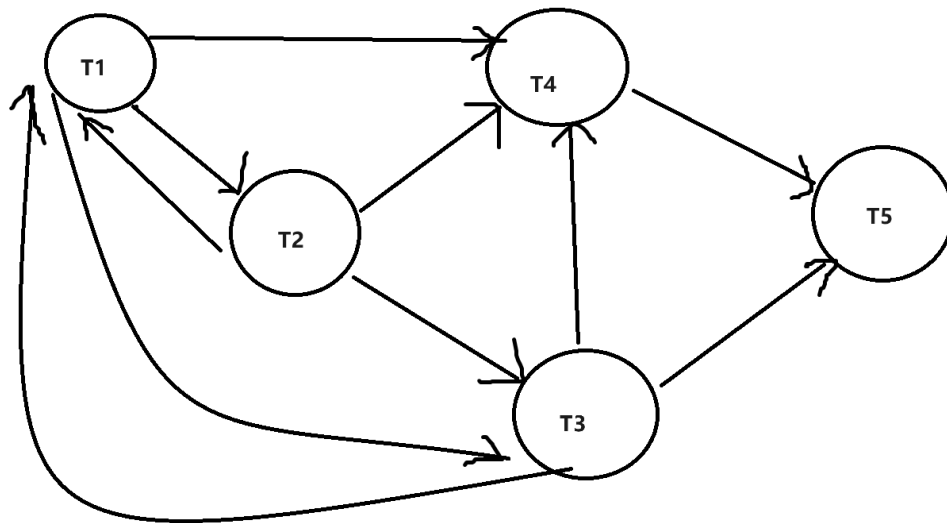
理财项目（NO，项目名，风险等级，收益类型）

四、计算题(15 分)

1. 设有如图所示调度 S, 判别 S 是否为冲突可串行化调度?如果是，则给出与 S 等价的一个串行调度。(5 分)

T1	T2	T3	T4	T5
R(A)				
				R(D)
	W(A)			
		R(A)		
W(A)				
			R(C)	
W(B)				
		W(A)		
			R(A)	
		R(C)		
			W(A)	
				R(C)
				W(C)

【解答】



做出优先图，可见图中有环，因此 S 不是冲突可串行化调度。

5. 设有关系模式 $R(A, B, C, D, E, F, G, H, I)$ ，其中各属性分别代表：

- A 交通事故编号；
- B 交通事故发生时间，
- C 交通事故具体原因，
- D 违章扣分，
- E 违章罚款金额，
- F 交通事故发车辆车牌号马，
- G 交通事故车型号及颜色，

H 违章驾驶员驾照号,

I 驾照已扣分数。

其中: (1) 司机与车辆之间的联系为 1:1, 车辆与违章之间的联系为 m:n;

(2) 一辆车只能有一个车牌号码, 且车牌号码唯一;

(3) 一个人只能有一个驾照, 且驾照号码唯一;

完成以下任务:

(1) 根据语义给出 R 的函数依赖; (5 分)

(2) 将该关系模式分解为 3NF。(5 分)

【解答】答: (1) $F\{H \rightarrow IF, F \rightarrow GH, A \rightarrow BC, AF \rightarrow DE\}$

(2) 求候选码

左边属性有 HFA

右边属性有 IFGHBCDE

所以左部属性是 A

右部属性是 BCDEIG

双部属性是 FH

那么 $AF^+ = ABCDEFGHI$

那么 $AH^+ = ABCDEFGHI$

其中 AF 已经在里面了, 所以不需要再加
得分解结果为 $\{HIF\}, \{FGH\}, \{ABC\}, \{AFDE\}$

五、应用题 (每小题 2 分, 共 10 分)

某连锁超市商品管理系统中, 包含如下关系模式:

门店 MD(门店编号 Mid, 门店名称 Mname, 店员人数 MRnum, 地址 Maddress)

商品 SP(商品编号 Pid, 商品名称 Pname, 价格 Price)

所属关系 MP(门店编号 Mid, 商品编号 Pid) 商品数量 Pnum)

1. 用关系代数表达式查询: 店员人数不少于 50 人的门店名称和地址;

$$\pi_{Mname, Maddress}(\sigma_{MRnum \geq 50}(MD))$$

2. 用关系代数表达式查询: 找出少供应了代号为“256”的商店所供应的全部商品的其它商店的商店名和地址:

$$\pi_{Mname, Maddress}(MD \bowtie (\sigma_{Pid < 256}(MP \div \pi_{Pid}(\sigma_{Mid=256}(MP)))))$$

3. 用 SQL 语句查询: 库存数量小于 10 件的商品名称及所在的门店名称:

SELECT SP. Pname, MD. Mname

```
FROM SP, MD, MP
WHERE SP.Pid = MP.Pid
AND MD.Mid = MP.Mid
AND MP.Pnum < 10;
```

4. 用 SQL 语句查询:各门店名称和所拥有的商品的总价格;

```
select distinct Mname,sum(Price*Pnum) as APrice
from sp natural join mp natural join md
group by Mname,Mid;
```

5. 将“256”号门店的“泉阳泉”的数量增加 2000。

```
update mp
set Pnum=Pnum+2000
where Mid=256 and Pid=(
select Pid
from sp
where Pname='泉阳泉'
) ;
```


第六套试题

2014 年

一、简答题

1. 数据库管理系统中 DDL 所能完成的操作包括哪些?

【解答】数据库管理系统中的 DDL (Data Definition Language, 数据定义语言) 主要负责数据的结构定义, 可以用来定义表、索引、约束、触发器等等。常用关键词: alter, drop, create.

2. 关系数据库设计中, 至少应满足的规范化条件是什么?

【解答】1NF.

3. 判断分解后的关系模式是否合理的两个重要标志是什么?

【解答】是否为无损分解和保持依赖。

4. 基于多表的视图, 可以完成哪些操作? 不能完成哪些操作?

【解答】于多表的视图可以完成的操作:

查询: 使用 SELECT 语句从视图中检索数据。

连接: 视图可以基于多个表的连接来创建, 允许跨表查询。

更新: 如果视图的查询不包含聚集函数、DISTINCT、GROUP BY 等限制, 可以更视图中的列。

插入: 在某些条件下, 可以向视图中插入新的行, 如果这些行符合底层表的约束。

删除: 可以从视图中删除行, 如果这些行在底层表中存在。

基于多表的视图不能完成的操作:

更新和删除限制: 如果视图包含以下元素, 则通常不允许更新或删除操作: 聚集函数、DISTINCT、GROUP BY、HAVING 子句。

修改视图结构: 不能直接修改视图的结构, 如添加或删除列。

直接修改底层表结构: 通过视图不能修改底层表的定义或结构。

违反底层表约束: 不能执行违反底层表约束 (如主键、唯一性约束) 的操作。

复杂的子查询操作: 不能在视图中使用复杂的子查询或嵌套子查询。

5. 实体之间的联系有哪几种? 分别举例说明?

【解答】一对一, 多对多, 多对一, 多对多。如一个班级里有三十名学生, 就是一对多。

6. 简述等值链接与自然连接的异同。

【解答】相同点:

两者都用于基于共同属性的值将两个表的记录进行配对。自然连接是等值链接的特例, 它自动在所有公共属性上执行等值链接, 并且简化结果集。

不同点:

等值链接需要指定连接条件。

自然连接隐含地在所有同名属性上应用等值链接条件, 无需指定条件, 自动使用表中所有共有的列名作为连接条件。

自然连接会去除结果中的重复列, 而等值链接会保留所有指定的连接列。

7. 简述 where 子句与 having 子句的区别。

【解答】where 子句用在 group by 之前, 作用于行; having 子句用于 group by 之后, 作用于分好的组。

8. 简述两阶段锁协议的具体含义

【解答】增长阶段：事务可以获得锁，但不能释放锁；缩减阶段，事务可以释放锁，但不能获得锁。

9. 简述数据库中为何要进行并发控制？

【解答】控制并发事务之间的并发影响，避免出现死锁、饥饿等影响数据库安全、一致性的情况，确保 ACID 特性。

10. 举例说明数据库中死锁含义及其解决方法？

【解答】数据库中的死锁是指两个或多个事务在执行过程中，因争夺资源而造成的一种相互等待的现象。在这种情况下，每个事务都在等待其他事务释放资源，而由于每个资源都被某个事务持有，导致没有事务能够继续向前推进，形成了一个循环等待的状态。解决方法是：破坏死锁的充要条件。

解决死锁的方法通常包括：

- **预防**：通过破坏死锁的四个必要条件之一或多个来避免死锁的发生。
- **检测**：允许死锁发生，然后通过某种机制检测到死锁，并采取措施解决。
- **避免**：使用算法动态地避免死锁的发生，例如银行家算法。
- **超时**：事务等待资源超过一定时间后超时，释放已占有的资源并回滚。

二、分析题(5*4=20)

1. 吉林大学的 BBS 系统，用户可以以游客的身份浏览其他人的留言贴，但是如果想要发言或者回复留言，则必须先登录方可留言，为什么？在数据库端如何实现？

【解答】游客身份只有 `select` 的浏览权限，没有输入输出权限。在数据库中，定义了游客角色和已登录用户的角色，并且分别赋予了他们不同权限。

2. 在从银行卡向学校一卡通转账的过程中，如果转账金额已经从银行卡扣除，但一卡通还未到账，机器死机无法正常工作了，请问数据库处于什么样的状态？这样的问题如何解决？

【解答】数据库处于不一致状态，解决方法：通过日志，进行事务回滚。

3. 支付宝账号通常与某一银行卡绑定，该卡不可透支，若目前卡内余额 100 元，另外一人使用绑定银行卡刷卡消费 100 元，能否支付成功？若不可以，在数据库端如何控制？

【解答】不能支付成功。数据库：采用可串行化控制。

4. 学校所有学生的数据都存在同一个表内，但各个学院的教务管理人员登陆后只能看到自己学院的学生，请问，在数据库端是如何实现的？

【解答】在数据库定义多个视图，每个视图仅存放对应学院的学生数据，同时个学院教务管理人员只被赋予了掌握自己学院视图的权利。

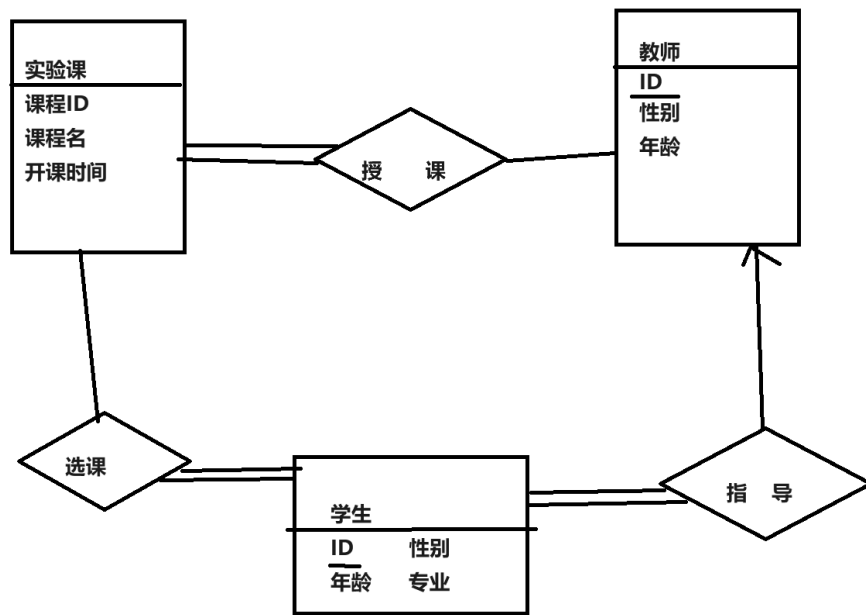
三、设计题(20 分)

吉林大学为了提高实验教学质量，为每门实验课配备了若干指导教师，指导学生实验。学院开设多门实验课程供学生选择，每个学生可以选择一到多门实验课，每门课程有若干指导教师，每个教师可以参与多门实验课的指导：在实验课中，一位教师负责指导多个学生，而一个学生有且必有一名指导教师。

1. 根据上述需求，画出 E-R 模型。

2. 将 E-R 模型转化为对应的关系模式，并说明每个关系模式中需要设置的完整性约束。（注：实体店属性根据实际情况自行设定，每个实体不少于三个属性，不多于五个属性。）

【解答】1.



2. 学生 (ID, 性别, 年龄, 专业) 主码是 ID
 教师 (ID, 性别, 年龄) 主码是 ID
 实验课 (课程 ID, 课程名, 开课时间) 主码是课程 ID
 选课 (学生 ID, 课程 ID)
 授课 (教师 ID, 课程 ID)
 指导 (教师 ID, 学生 ID)

四、应用题

1. 有仓库-物料存储系统，其关系模式如下：

W(Wid, WN, Addr); 分别表示仓库(仓库号, 仓库名称, 地址)

M(Mid, MN, Price, p_Time, q_Time): 分别表示(物料号, 物料名称, 价格, 生产日期, 保质期)

ST(Wid, Mid, SNum); 分别表示(仓库号, 物料号, 存储数量)

请完成如下查询:(10 分)

5) 用关系代数表达式, 查询存储了全部物料的仓库名称。

$$\Pi_{WN}(W \bowtie \Pi_{Wid}(ST \div \Pi_{Mid}(M)))$$

6) 用关系代数表达式, 查询所有存储数量高于 100 的物料号。

$$\Pi_{Mid}(\sigma_{Num > 100}(\rho_A(Mid, Num)(Mid g_{sum(sNum)}(ST))))$$

7) 用 SQL 语句, 查询所有没有生产日期的物料号和物料名称。

```
SELECT Mid, MN
FROM M
WHERE p_Time IS NULL;
```

8) 用 SQL 语句, 建立视图 V, 统计截止到今天, 还有三个月就要到保质期的物料号和物料名称。

```
CREATE VIEW V AS
SELECT Mid, MN
FROM M
WHERE q_Time <= CURRENT_DATE + INTERVAL '3 MONTH';
```

5) 用 SQL 语句, 修改物料名为“海绵”的库存, 增加 100 件。

```
UPDATE ST
SET SNum = SNum + 100
WHERE Mid = (SELECT Mid FROM M WHERE MN = '海绵');
```

2. 设有关系模式 **R**(A, B, C, D, E, F, G)

其函数依赖集 **F**={**AB**→**C**, **B**→**DE**, **C**→**G**, **G**→**A**}

求出模式 R 的所有候选码) (7 分)

【解答】

F 中所有在左边的属性有 **ABCG**

所有在右边的属性有 **CDEGA**

那么可以知道:

左部属性 **B**

右部属性 **DE**

双部属性 ACG

外部属性 F

首先计算 $(BF)^+ = \{BFDE\}$

然后计算 ABF^+ 、 CBF^+ 、 GBF^+

第一个可以推 $\{ABCDEFGF\}$

第二个可以推 $\{ABCDEFGF\}$

第三个可以推 $\{ABCDEFGF\}$

因此候选码为：ABF, CBF, GBF

3, 设有关系 R 和函数依赖 F:

其中 $R(A, B, C, D, E)$, $F(ABC \rightarrow DE, BC \rightarrow D, D \rightarrow E)$

请将关系 R 逐步分解为 3NF。(8 分)

【解答】首先使用合并律, 发现不需要合并

然后尝试去除无关属性

$ABC \rightarrow DE$ 中 A 可以去除, 因为 $BC \rightarrow D, D \rightarrow E$, 可知 $BC \rightarrow DE$

然后 $BC \rightarrow D$ 可以去除, 因为已经有 $BC \rightarrow DE$

$BC \rightarrow DE$ 中 E 可以去除, 因为 $D \rightarrow E$

最终得 $F_c = \{BC \rightarrow D, D \rightarrow E\}$

求得 R 的主码, 接下来求候选码;

F 左边有属性 ABCD

右边有属性 DE;

左部属性 ABC

右部属性 E

双部属性 D

$$\{ABC\}^+ = \{ABCDE\}$$

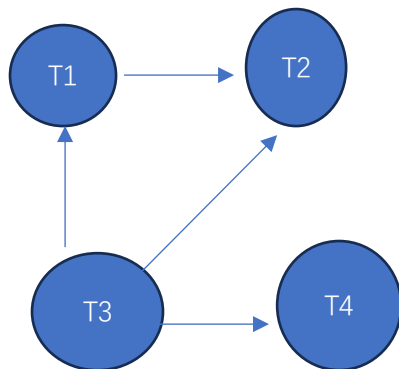
ABC 就是候选码, 不在 BCD 和 DE 中, 加上

因此可以分解为 $\{ABC\}\{BCD\}\{DE\}$

4. 设有如图所示调度 S，判断 S 是否为冲突可串行化调度？如果是，则给出与 S 等价的一个串行调度。（5 分）

T1	T2	T3	T4
R(A)			
			R(D)
W(A)			
	R(A)		
		R(B)	
		W(B)	
	W(A)		
		R(C)	
			R(C)
			W(C)
W(B)			
	R(B)		
		R(D)	

【解答】是冲突可串行化调度。原因如下：
做出调度优先图：



可以看出，优先图中是没有环的，因此是冲突可串行化调度。

与 S 等价的一个可串行调度其实就是该图的一个拓扑序列，如：T3 → T1 → T4 → T2

第七套试题

2008 级《数据库系统原理》考试试题(B 卷)

一、[13 分]根据自然语言描述绘制某研究院的 E-R 图：

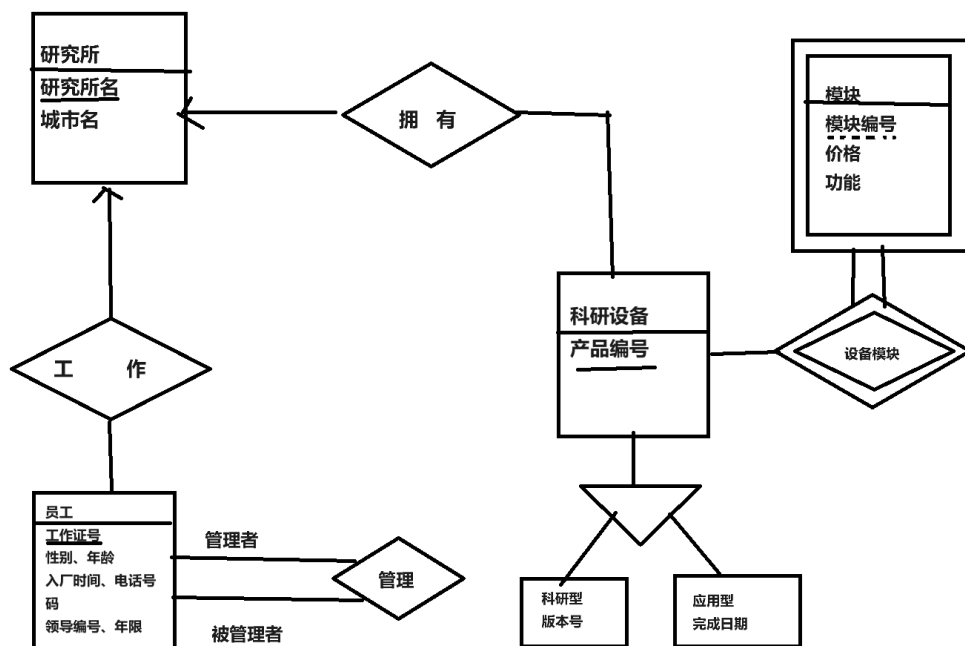
(1) 研究院由多个研究所组成，每个研究所由唯一的的名字标识，位于某个城市，研究院管理每个研究所的资产。

(2) 研究所的员工由其工作证号来标识，还需要记录每个员工的姓名、电话号码(包括办公电话和手机电话)及其直属领导的工作证号。研究院还需要知道员工开始工作的日期，由此日期可以推知员工的工作年限。

(3) 研究所开发的产品各不相同，分别赋以不同的产品编号。每种产品均分为科研型和应用型两种。科研型产品应记录版本号，应用型产品应记录该产品的完成日期。

(4) 每个产品由若干个模块构成。每个模块用模块号标识。研究院需要记录每个模块的具体功能和价格。虽然不同产品的模块号可能重复，但同一产品的模块号是不重复的。

【解答】



二、[14 分，每小题 7 分]关于关系模式 $R=(A, B, C, D, E)$ 的函数依赖集 F 如下所示：

$F=\{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$

(1) 计算正则覆盖 F_c . (2) 计算闭包 $(AB)^+$

【解答】 $F_c=\{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$

$(AB)^+=ABCDE$

三、[16 分，每小题 4 分]考虑下图所示员工数据库。为下面每个查询语句写出 SQL 表达式

Employee(employee-name, street, city)
 works(employee-name, company-name, salary)
 company(company-name, city)

1. 找出所有为 First Bank Corporation 工作的员工的名字。

```
SELECT employee-name FROM works
WHERE company-name = 'First Bank Corporation';
```

2. 修改数据库，使得 Jones 现在居住在 Newtown 市。

```
UPDATE Employee
SET city = 'Newtown'
WHERE employee-name = 'Jones';
```

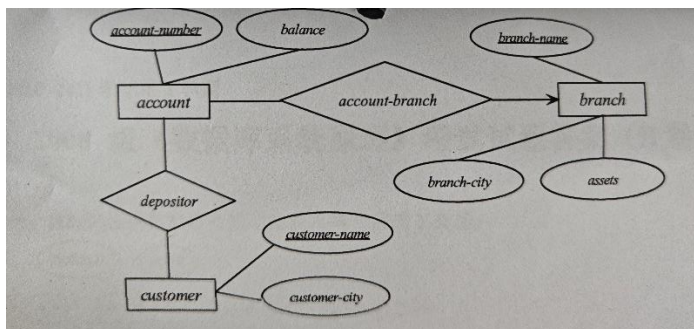
3. 找出各个公司员工的平均工资，并按照公司名称排序(逆序)。

```
SELECT company-name, AVG(salary) AS average_salary
FROM works
GROUP BY company-name
ORDER BY company-name DESC;
```

4. 为 First Bank Corporation 所有员工增加 10%的薪水。

```
UPDATE works
SET salary = salary * 1.10
WHERE company-name = 'First Bank Corporation';
```

四、[12 分]根据下面的 E-R 图设计关系数据库,要求指出相应的主键和外键。



【解答】

account (account_number, balance, branch name) primary key (account_number)
 foreign key (branch_name)
 branch (branch_name, branch_city, assets) primary key (branch_name)
 customer (customer_name, customer_city) primary key (customer_name)
 depositor (account_number, customer_name)
 primary key (account_number, customer_name)
 foreign key (account number)
 foreign key (customer_name)

五、[15 分, 每小题 5 分]设有关系数据库:

d (d#, dn, dx, da, dt, s#)

p (p#, pn, px, w#)

dp (d#, p#, wa)

s (s#, sn, sl)

(1) d#、dn、dx、da、dt 依次分别表示医生的编号、姓名、性别、年龄、职称；

(2) p#、pn 和 px 依次分别表示住院患者的编号、姓名和性别

(3) s#、sn 和 sl 依次分别表示医院科室的编号、名称和地址；

(4) w#表示病房编号；• wa 表示工作量；

(5) 关系 dp 表示医生治疗患者的联系。

试用关系代数表达如下要求：

1. 求职称为 prof 的医生的姓名和年龄。

【解答】 $\Pi_{dn,da}(\sigma_{dt='prof'}(d))$

2. 求姓名为 wang 的医生治疗的患者的编号和姓名。

【解答】 $\Pi_{p#,pn}(\sigma_{dt='wang'}(d \bowtie dp))$

3. 求治疗 w2 号病房的所有患者的男(用 m 表示)医生的编号

【解答】 $\Pi_{p#,d\#}(d \bowtie dp \bowtie p) \div \Pi_{p\#}(\sigma_{w\#='w2' \text{ and } px='m'}(p))$

六、[12 分]试判断下图所示 5 个事务满足冲突可串行化，给出相应的串行化调度次序，并解释理由

T1	T2	T3	T4	T5
R(Y)				
	W(Y)			
W(Y)				
		R(Y)		
			R(Z)	
R(X)				
		W(Y)		
			R(X)	
			W(X)	
		W(Z)		
				R(Z)
				W(Z)

【解答】

T1 和 T2 两个事务形成环，不满足冲突可串行化 (6 分)

T2 指令放到最前方 T2, T1, T3, T4, T5 (6 分)

七、[18 分，每小题 6 分]问答题：

1、说明视图与关系表的区别与联系。

【解答】

视图 (View) 与关系表 (Table) 在数据库中都是用来存储数据的方式，但它们之间存在一些关键的区别和联系：

区别：

1. **存储方式**：关系表实际存储数据在数据库中，而视图是基于表的查询结果的虚拟表示，不存储数据。
2. **更新能力**：关系表可以直接更新（插入、更新、删除行），视图通常不可更新，除非满足特定条件（如简单视图，不包含聚集函数、DISTINCT 等）。
3. **性能**：表可以直接访问数据，视图则需要每次进行查询解析，可能会有性能开销。
4. **安全性**：视图可以作为数据的逻辑封装，限制用户直接访问底层表，增强安全性。

联系：

1. **结构**：视图和表都可以有行和列，可以进行 SELECT 查询操作。
2. **数据操作**：可以通过视图间接操作表中的数据，如果视图可更新。
3. **依赖性**：视图依赖于其定义中引用的表，表结构变化可能影响视图。
4. **查询**：视图可以通过 SQL 查询定义，查询结果可以是来自一个或多个表的数据。

视图提供了一种方便的方式来访问和操作表中的数据，同时提供了逻辑上的封装和安全性。

2、举例说明强实体集和弱实体集的区别和联系。

【解答】假设有一个学校数据库系统，其中包含以下实体集：

学生 (Student)：这是一个强实体集，因为学生可以独立存在，拥有自己的主键（如学号）。

学生课程成绩 (StudentCourseGrade)：这是一个弱实体集，因为它依赖于两个实体集——学生 (Student) 和课程 (Course)。学生课程成绩不能独立存在，它需要知道是哪个学生 (Student 的学号) 以及哪门课程 (Course 的课程号)，并且通常使用这两个强实体集的属性组合作为它的复合主键。

在这个例子中，学生课程成绩的记录必须与存在的学生和课程相关联，展示了强实体集和弱实体集之间的依赖性和联系。

3、举例说明事务的 ACID 特征。

【解答】事务的 ACID 特性指的是原子性 (Atomicity)、一致性 (Consistency)、隔离性 (Isolation) 和持久性 (Durability)。以下是每个特性的举例说明：

原子性 (Atomicity)

— 例子：在银行转账中，从一个账户扣除一定金额并存入另一账户的操作是原子性的。如果第一个账户的扣款成功，但第二个账户的存款失败，整个事务将回滚，就像这两个操作从未发生一样，确保了事务的完整性。

一致性 (Consistency)

— 例子：在线购物平台在处理订单时，库存数量和订单状态需要保持一致。如果一个客户下单购买了一个产品，系统会检查库存是否充足。如果库存不足，订单不会被创建，系统会提示客户产品不可用，这保证了数据库从一个一致的状态转移到另一个一致的状态。

隔离性 (Isolation)

— 例子：假设两个用户同时进行网上银行转账操作，事务的隔离性确保了这两个操作不会互相干扰。即使两个事务并发执行，每个事务都像在独立执行一样，不会看到其他事务的中间状态，从而避免了数据的不一致性。

持久性 (Durability)

— 例子：一旦银行转账事务提交，即使系统发生故障，转账记录也不会丢失。系统会将事务结果写入到持久化存储中，如数据库或日志文件，确保了即使在系统崩溃后，这些更改也不会丢失。

这些特性共同确保了数据库管理系统在并发操作和系统故障的情况下,仍能保持数据的准确性和可靠性。